

微型指针

Michael A. Bender Alex Conway Martín Farach-Colton William Kuszmaul
Stony Brook University, VMware Research Group, Rutgers University, Massachusetts

arXiv:2111.12800v1 [cs.DS] 2021 年 11 月 24 日

2026 年 7 月 15 日

摘要

本文引入一个新的数据结构对象，我们称之为**微型指针**（tiny pointer）。在许多应用中，传统的 $\log n$ 比特指针可以被 $o(\log n)$ 比特的微型指针替代，仅以常数因子时间开销为代价。我们发展了微型指针的全面理论，并给出了**固定大小微型指针**（即所有微型指针必须相同大小的设置）和**可变大小微型指针**（即平均微型指针大小必须很小，但某些微型指针可以更大的设置）的最优构造。如果一个微型指针引用一个填充到负载因子 $1 - 1/k$ 的数组中的元素，则在固定大小情况下最优微型指针大小为 $\Theta(\log \log \log n + \log k)$ 比特，在可变大小情况下为 $\Theta(\log k)$ 期望比特。我们的微型指针构造还要求我们重新审视几个与球和桶相关的经典问题；这些结果可能具有独立的意义。

使用微型指针，我们重新审视五个经典的数据结构问题。我们证明：

- 一个为 n 个键存储 n 个 v 比特值、支持常数时间修改/查询的数据结构可以实现为占用 $nv + O(n \log^{(r)} n)$ 比特空间（对任意常数 $r > 0$ ），只要用户为每个键存储一个期望大小为 $O(1)$ 的微型指针——这里 $\log^{(r)} n$ 是第 r 次迭代对数。
- 任何二叉搜索树可以被做成简洁的，具有常数因子时间开销，如果允许 $O(\log n)$ 时间的修改，甚至可以达到距离最优 $O(n)$ 比特以内——这对基于旋转的树（如 splay 树和红黑树）也成立。
- 任何固定容量键值字典可以被做成**稳定的**（即项一旦插入就不移动），具有常数时间开销和 $1 + o(1)$ 空间开销。
- 任何要求统一大小值的键值字典可以被做成支持任意大小值，具有常数时间开销，并且每个 j 比特值仅额外消耗 $\log^{(r)} n + O(\log j)$ 比特空间（ $r > 0$ 为任意常数）。
- 给定一个大小为 $(1 + \epsilon)n$ 的外部内存数组 A ，包含最多 n 个键值对的动态集合，可以维护一个大小为 $O(n \log \epsilon^{-1})$ 比特的内部内存暂存区（stash），使得任何键值对在 A 中的位置可以在常数时间内计算（且无 I/O）。

这些都是被广泛研究的经典问题，在每种情况下，微型指针都允许我们采用使用指针的自然空间低效解决方案，并免费使其空间高效。

1 1 引言

存储一个指针需要多少比特？如果我们对指针一无所知，只知道它引用大小为 n 的数组中的一个元素，那么存在 $\log n$ 比特的下界。

然而，对于指针的许多（可能甚至大多数）用途，这个信息论下界并不适用。正如我们将在本文中看到的，即使关于指针的少量先验信息（例如，节点在链表中的前驱）也可以用来突破 $\log n$ 下界。

本文引入一个通用工具，我们称之为**微型指针**（tiny pointer），用于压缩指针。在使用指针的设置中，微型指针通常可以用来替代指针，从而消除指针的几乎所有空间开销。

什么是微型指针？ 假设 n 个或更多的用户（即 Alice、Bob 等）共享一个大小为 n 的数组 A 。用户可以通过函数 $\text{Allocate}()$ 请求 A 中的一个位置，该函数返回一个指向现在为该用户独占保留的位置的指针 p （如果有可用位置）；用户稍后可以通过调用函数 $\text{Free}(p)$ 放弃该内存位置。每个用户承诺一次最多分配一个内存位置¹。例如，如果 Alice 调用 $\text{Allocate}()$ 获得指针 p ，她必须在再次调用 $\text{Allocate}()$ 之前调用 $\text{Free}(p)$ 。

¹ 用户 k 可以通过为每次分配创建一个唯一标签来请求多个位置。在这种情况下，我们简单地将该次分配的“用户”视为拼接 $k \circ \ell$ ，因此用户 k 可以有多个分配而不违反唯一性要求。

指针 p 需要多大？自然的答案是每个指针使用 $\log n$ 比特。然而，每个指针有不同所有者这一事实使得将指针压缩到 $o(\log n)$ 比特成为可能。一个关键的洞察是，**同一个指针 p 对不同用户可以有不同的含义**，通过以下方案：用户 k 可以调用 $\text{Allocate}(k)$ 来获得一个微型指针 p ；他们可以通过计算一个函数 $\text{Dereference}(k, p)$ （其值仅依赖于 k 、 p 和随机比特）来解引用微型指针 p ；他们可以通过调用函数 $\text{Free}(k, p)$ 来释放微型指针 p 。

微型指针不受 $\log n$ 比特信息论下界约束的原因是， k 和 p **一起**编码了分配的位置，而不仅仅是 p 单独。因此，该方案提供了一种机制，用于如何利用关于指针的已有信息（即谁“拥有”该指针）将指针压缩到 $o(\log n)$ 比特大小。

我们将函数 $\text{Allocate}(k)/\text{Dereference}(k, p)/\text{Free}(k, p)$ 的算法，连同数组 A 和任何关联的元数据 M ，称为**解引用表**（dereference table）。我们经常将用户（即微型指针的所有者）称为**键**，将存储在微型指针指向的分配位置的数据称为**值**。一个在 nb 比特数组中存储 b 比特值（并使用 $O(n)$ 比特元数据）的解引用表被称为支持**负载因子** $1 - \delta$ ，如果该表能够一次存储 $(1 - \delta)n$ 个值。

理想的解引用表将同时支持 $\delta = o(1)$ 的负载因子、 $o(\log n)$ 的微型指针大小和常数时间操作。正如我们稍后将讨论的，我们证明了最佳可实现的负载因子 $1 - \delta$ 与最佳可实现的微型指针大小 s 之间的权衡曲线。构造此权衡曲线上的最优解引用表是本文的核心问题之一。

使用微型指针获得微型数据结构。除了构造具有微型指针的解引用表，我们还展示

了这样的解引用表可以用于获得若干经典问题的改进解决方案：

- 一个为 n 个键存储 n 个 v 比特值、支持常数时间修改和查询的数据结构可以实现为占用 $nv + O(n \log^{(r)} n)$ 比特空间，对任意常数 $r > 0$ ，只要用户为每个键存储一个期望大小为 $O(1)$ 的微型指针——这里 $\log^{(r)} n$ 是第 r 次迭代对数。²

² 即 $\log^{(1)} n := \log n$ 且 $\log^{(i+1)} n := \log \log^{(i)} n$ 。

- 任何在 n 个节点中存储 n 个可排序键的二叉搜索树可以被做成简洁的，具有常数时间开销，并且如果允许 $O(\log n)$ 时间的修改，可以在 $O(n)$ 比特最优范围内。这对基于旋转的树（如 splay 树，被猜想为动态最优的）也成立。
- 任何存储 v 比特值的固定容量键值字典可以被做成**稳定的**（即项一旦插入就不移动），具有常数时间开销和每个值额外的 $O(\log v)$ 比特空间开销。
- 任何要求统一大小值的键值字典可以被做成支持任意大小值，具有常数时间开销，并且每个 j 比特值仅额外消耗 $\log^{(r)} n + O(\log j)$ 比特空间（ $r > 0$ 为任意常数）。
- 给定一个大小为 $(1 + \epsilon)n$ 的外部内存数组 A ，包含最多 n 个键值对的动态集合，可以维护一个大小为 $O(n \log \epsilon^{-1})$ 比特的内部内存暂存区（stash），使得任何键值对在 A 中的位置可以在常数时间内计算（且无 I/O）。

这些问题的共同点是，每个问题都可以使用指针以空间低效的方式轻松解决，而空间高效地解决它们的困难源于消除指针开销的挑战。

贯穿我们对微型指针的各种应用，一个主题是拥有完整的最优微型指针构造权衡曲线的重要性。这是因为需要在两种类型的空间开销之间取得平衡：存储微型指针本身的开销，以及存储解引用表的开销。前者由微型指针大小决定，后者由负载因子决定。

本文。在本文中，我们首先发展微型指针的全面理论。我们考虑**固定大小微型指针**（其中每个微型指针具有有界大小）和**可变大小微型指针**（其中每个微型指针具有有界期望大小）。对于两种类型的微型指针，我们确定了在解引用表中负载因子与微型指针大小之间的最优权衡曲线。然后，我们继续介绍上述微型指针的五个应用。作为辅助结果，我们还将微型指针构造重新解释为球与桶的结果。通过这样做，我们在一些重要参数范围内改进了动态负载均衡的已知界限。

1.1 1.1 结果：构造最优微型指针

在第 3、4 和 5 节中，我们为微型指针大小 s 与解引用表负载因子 $1 - \delta$ 之间最佳可实现的权衡曲线建立了紧的渐近界。

固定大小和可变大小微型指针的最优权衡。对于固定大小微型指针，我们证明对于任何负载因子 $1 - \delta = 1 - o(1)$ ，微型指针大小 s 存在 $\Omega(\log \log \log n)$ 的下界。另一方面，

以 δ 为参数，我们证明可以实现固定微型指针大小 $s = O(\log \log \log n + \log \delta^{-1})$ ，并给出下界证明此权衡曲线是紧的。

我们证明 $\log \log \log n$ 障碍可以通过使用可变大小微型指针来消除。我们证明对于任何负载因子 $1 - \delta$ ，可以实现平均微型指针大小 $s = O(1 + \log \delta^{-1})$ ，并再次证明此权衡曲线对所有 δ 是紧的。

对于可变大小微型指针，我们的构造提供了每个微型指针大小的非常强的集中界：如果期望大小为 k ，则任何给定分配返回大小大于 $k + j$ ($j > 0$) 的微型指针的概率关于 j 是双重指数小的。

我们所有的解引用表构造保证以高概率（即以概率 $1 - 1/\text{poly}(n)$ ）实现常数时间操作。因此，微型指针可以集成到数据结构中，仅引入常数因子时间开销。

与球和桶的关系。在第 7 节中，我们将微型指针结果重新解释为球到桶的结果。值得注意的是，我们能够将我们的技术应用于动态负载均衡问题，其中有 n 个桶且一次最多有 $m = hn$ 个球存在：对于 $h \geq 1$ ，我们给出一个使用 $d + 1$ 个哈希函数的球与桶方案，实现最大负载 $h + O\left(\sqrt{h \log(hd)}\right) + \frac{\log \log n}{d \log d}$ ，这在 $hd = o(\log n)$ 时显著改进了现有的技术水平 [55, 56]。

为了理解解引用表与球与桶方案之间的关系，将键视为必须分配到不同桶中的球。每个球 x 有某个探测序列 $h_1(x), h_2(x), \dots \in [n]$ ，表示它可以被放置的桶。支持大小为 $O(s)$ 的微型指针等价于维护一个动态的球到桶分配，使得每个球 x 在某个满足 $i \leq 2^{O(s)}$ 的桶 $h_i(x)$ 中。

使得这个球与桶问题有趣的是，同一个球可以随时间被插入、删除并随后重新插入。第一次插入球时，其探测序列 $h_1(x), h_2(x), \dots$ 与解引用表的状态独立。但如果球被删除后又重新插入，则情况不再如此：解引用表的状态现在已被探测序列影响（并且部分是探测序列的函数）。结果是，在这个完全动态的设置中，即使是非常简单的球与桶方案（例如随机探测 [40] 或线性探测 [39, 52]）的行为也抵抗了理论分析。³

³ 此设置中的工作通常将线性探测和随机探测视为构建开放寻址哈希表的技术。在球放置后不能移动的设置中（或等价地，哈希表删除通过墓碑实现），随机探测或线性探测的唯一已知界限归功于 Larson [40]，他分析了随机插入/删除下的随机探测。

构造小型微型指针的一个关键洞察是，通过设计每个“球”的探测序列具有某种仔细的结构，我们可以为任意的球插入和删除序列实现小的探测复杂度（从而小的微型指针）。同样的技术也使我们能够重新审视其他相关问题，例如容量无界的桶中的动态负载均衡。

1.2 1.2 结果：五个数据结构应用

我们现在更详细地描述微型指针的五个应用。第一个应用是经典的为与键关联的值存储动态集合的数据结构问题。接下来的三个应用是黑盒变换，我们展示如何从大类数据结构中消除空间低效性。最后一个应用是外部内存存储中一个经典问题的新数据结

构。

克服数据检索的 $\Omega(\log \log n)$ 比特下界。我们的第一个应用重新审视经典的数据检索问题 [5, 26–28]，其中一个数据结构必须为某个集合 S 中每个 k 比特键存储一个 v 比特值，并且必须回答检索与给定键关联的值的查询⁴。在静态情况下（键/值预先给定），可以使用 $nv + O(\log n)$ 比特空间以 $O(1)$ 时间查询解决检索问题 [27, 28]；但在动态情况下（键/值随时间插入/删除，且一次最多有 n 个键/值对，键取自某个大的多项式大小全空间），已知检索问题的任何解必须使用 $nv + \Omega(n \log \log n)$ 比特空间的下界，即使允许超常数时间操作 [5, 26]。这意味着每个值的元数据比特数平均为 $\Omega(\log \log n)$ ，即使值的大小为 $v = o(\log \log n)$ 。

⁴ 注意，查询要求键 $x \in S$ ——如果 $x \notin S$ ，数据结构允许返回任意值。

我们证明，通过稍微修改检索问题的规范，我们可以完全消除 $\Omega(\log \log n)$ 的每项浪费比特下界。特别地，假设每当用户插入一个键/值对 (x, y) 时，他们获得一个他们负责存储的小提示 h 。（我们将保证该提示具有常数期望大小。）将来，当用户希望恢复 x 的值 y 时，他们同时向检索数据结构提供键 x 和提示 h 。我们称此为**放松检索问题** (relaxed retrieval problem)，并将这些提示称为**微型检索器** (tiny retrievers)。

放松检索问题也可以看作是微型指针问题的放松：微型检索器 h 类似于微型指针，除了对 (x, h) 不必完全编码 y 的位置——相反，放松检索数据结构可以利用的不只是 x 和 h ，还可以利用一个小的辅助数据结构，其目的是帮助恢复 y 。

鉴于我们已经陈述了微型指针的紧界，人们可能会倾向于假设相同的界应该对微型检索器也成立。我们发现并非如此。我们展示了如何构造期望大小 $O(1)$ 的微型检索器，同时以高概率支持常数时间查询，并允许以下权衡曲线：插入/删除使用 $O(r)$ 时间，数据结构的大小变为 $nv + O(n(1 + \log^{(r)} n))$ 比特。因此，使用常数时间操作，我们可以实现例如 $nv + O(n \log \log \log \log \log n)$ 的大小；使用 $O(\log^* n)$ 时间操作，我们可以实现 $nv + O(n)$ 的大小。此外，在值长度 v 为亚对数的特殊情况下（满足 $v \leq \frac{\log n}{\log^{(r)} n}$ ），即使对常数 r ，空间消耗也降至 $nv + O(n)$ 比特。

值得注意的是，我们对微型检索器的构造本身就是微型指针的一个直接应用——事实上，微型检索器就是期望大小为 $O(1)$ 的可变长度微型指针。这是因为能够构造指向具有 $\Omega(n)$ 个条目的数组的 $O(1)$ 长度微型指针，最终使我们能够将放松检索问题归约到字典问题，而对后者已知存在高度空间高效的解 [13]。

我们指出，微型指针和微型检索器之间的区别在我们下面的几个应用中最终是显著的。在某些情况下，微型检索器提供了一条通往显著的（且意想不到的）空间效率的路径，而在其他情况下，微型指针提供的平滑权衡曲线和类似指针的行为使它们更合适。微型检索器的优势在于它们提供了时间与空间之间的陡峭权衡；微型指针的优势在于它们提供了对元素的无间接引用，以及不同类型空间消耗（指针大小和负载因子）之间的灵活权衡。

简洁的基于旋转的二叉搜索树。为描述我们的第二个应用，我们首先进入简洁二叉

树的世界。由于 n 个节点上最多有 4^n 棵有序二叉树，二叉树的指针结构可以在 $O(n)$ 比特中编码。这一观察导致了关于二叉树最优（和近最优）编码的大量工作 [16, 25, 32, 34, 45, 46, 49, 51]。除了导航外，最先进的树还支持各种各样的查询操作（例如子树大小 [16, 32, 45, 46, 51]、深度 [16, 46]、最低公共祖先 [16, 46]、层级祖先 [16, 46] 等），同时也支持基本的动态性（例如插入/删除叶子 [16, 32, 45, 46, 51]、在边的中间插入节点 [16, 32, 45, 46, 51]、压缩长度为二的路径 [16, 32, 45, 46, 51] 等）。

然而，一种自然的动态操作形式已被证明是难以捉摸的：已知的简洁二叉树不能高效地支持**旋转**。缺乏对旋转的支持对于二叉搜索树尤其重要，它们在 n 个节点中存储 n 个可排序键。几乎所有动态平衡二叉搜索树（例如 AVL 树 [4]、红黑树 [36]、splay 树 [54]、treap [6, 53] 等）在修改树时都依赖旋转。这些树结构都无法用已知的简洁树技术编码。

我们给出了一种将动态二叉搜索树转换为简洁数据结构的随机化黑盒方法。如果简洁搜索树中有 n 个键，每个 k 比特长，则简洁搜索树的大小将为 $nk + O(n \log^{(r)} n)$ 比特。该变换仅对查询操作引入常数因子时间开销，对树的修改仅引入 $O(r)$ 因子时间开销。例如，如果我们设 $r = O(\log n)$ ，则边遍历花费 $O(1)$ 时间，边插入/删除花费 $O(\log n)$ 时间，树结构使用 $O(n)$ 编码。相比之下，之前在空间高效二叉搜索树中实现旋转的现有最佳技术 [46] 也将树结构编码为 $O(n)$ 比特（实际上是 $2n + o(n)$ 比特），但实现单次旋转需要 $\Omega(\log n)$ 时间。

当 r 设为 $O(1)$ 时，运行时间被保持这一事实意味着其他性质，如动态最优性，也被保持。例如，如果 splay 树 [54] 是动态最优的（如广泛相信的动态最优性猜想 [54] 所假设的），那么简洁 splay 树也是动态最优的。

空间高效的稳定字典。我们的第三个应用是一种将任何固定容量键值字典转换为**稳定字典**的黑盒方法，具有相同的操作集和仅常数因子时间开销。如果原始字典存储 v 比特值，则新稳定字典也存储 v 比特值，并且每个值使用的额外空间最多比原始数据结构多 $O(\log v)$ 比特。

形式上，一个键值字典（例如二叉搜索树、哈希表等）是**稳定的**，如果每当插入一个键值对时，值被存储的位置永不改变。（此性质有时也称为**引用完整性** [52] 或**值稳定性** [11]。）稳定性确保用户可以维护指向数据结构的指针，而不因数据结构的变化而使这些指针失效 [37, 52]。稳定性是许多库数据结构中的严格要求 [17–24]（这也是为什么高性能语言如 C++ 使用链式哈希 [17, 18]——它是稳定的——而不是使用更节省空间的替代方案，如线性探测 [38, 50] 或 cuckoo 哈希 [29, 33, 48] 的核心原因）。

关于在空间高效哈希表中实现稳定性的实证研究可追溯到 20 世纪 80 年代 [37, 52]（另见 Knuth 第 3 卷中的讨论 [39]），由此产生的技术已被构建到 Google [3] 和 Facebook [31] 发布的广泛使用的哈希表中。在理论方面，如果数据结构存储 k 比特键和 v 比特值（ $k, v = O(\log n)$ ），已知可以以每个值额外 $\Theta(\log \log n)$ 比特空间的代价实现稳定性 [26]，但不知道每个值 $\Omega(\log \log n)$ 比特是否必要。⁵ 我们的结果表明并非如此——稳定

性可以每个值仅用 $O(\log v)$ 额外比特即可实现。这在值大小 v 很小的情况下尤为值得注意⁶。我们的结果适用于任意固定容量字典，包括例如上面构造的简洁 splay 树。

⁵ 有趣的是，对于几种特定方法，已知每个值 $\Omega(\log \log n)$ 比特是必要的，例如如果稳定性通过完美哈希实现（见 [26] 的定理 2）。

⁶ 一个特别引人注目的推论如下：如果我们希望为数据结构中的每个键存储 $O(1)$ 个控制比特，并且我们希望这些比特的位置是稳定的，以便无权访问数据结构的第三方仍然可以访问/修改控制比特，那么我们可以仅使用每项 $O(1)$ 额外比特空间来实现。

具有可变大小值的空间高效字典。我们的第四个应用是一种将任何键值字典（设计用于存储固定大小值）转换为可以存储不同键的不同大小值的字典的黑盒方法。由此产生的数据结构引入常数因子时间开销，并提供以下关于空间效率的保证。令 $\log^{(r)} n$ 为第 r 次迭代对数，并设 r 为我们选择的一个正常数。新数据结构为每个值 x 仅增加 $O(\log^{(r)} n + \log |x|)$ 比特的额外空间开销。（有趣的是，此应用中的迭代对数 $\log^{(r)} n$ 与我们之前应用中的来源完全不同。）

存储可变长度值的能力还产生了多集问题的一个简单解，即如何设计一个存储键的多集（而不仅仅是集合）的空间高效常数时间哈希表。多集问题首先由 Arbitman 等人 [8] 作为开放问题提出，他们给出了一个能够存储集合但不能存储多集的简洁常数时间哈希表。随后一系列工作给出了多集问题的解，首先是在随机多集的情况下 [14]，然后最近对任意多集 [15]。然而，已知解决方案有一个缺点：重复键的空间界与不重复键的相同。因此，如果某个键有 m_i 个拷贝，则它们被允许占用单个拷贝的 m_i 倍空间，尽管原则上 $m_i - 1$ 个拷贝可以使用 $\lceil \log m_i \rceil$ 比特计数器编码。我们的变换给出了一个避免此缺点的简单替代方案，甚至可以直接应用于 Arbitman 等人的原始哈希表 [7]：通过将每个键的重数存储为（可变长度）值，可以支持任意多集，每个键仅额外花费 $\log^{(r)} n + \log m_i + O(\log \log m_i)$ 比特空间代价，其中 m_i 是键的重数， r 是我们选择的一个正常数；考虑到仅存储重数就需要 $\log m_i$ 比特，这是非常节省空间的。我们解决方案的一个优点在于，它也可以直接应用于其他字典，例如本节前面讨论的简洁 splay 树。

最优内部内存暂存区。我们的最后一个微型指针应用重新审视外部内存数据结构中最古老的问题之一：维护一个小型内部内存暂存区（stash），允许直接定位元素在大型外部内存数组中的位置。

更详细地说，该问题可以描述如下 [35]。我们给定一个（最初为空的）具有 $(1 + \epsilon)n$ 个槽位的外部内存数组，参数为 ϵ, n 。我们必须维护一个动态变化的键值对集合 S （键是唯一的），使得每次键值对 (x, y) 被插入 S 时，对 (x, y) 被分配到外部内存数组中的某个永久位置。然后，我们还必须维护一个小型内部内存数据结构 X ，称为**暂存区**，可用于为每个键 x 恢复其键值对 (x, y) 精确存储在外部内存中的位置。暂存区使得查询可以通过**对外部内存的单次访问**完成。

关于设计空间高效和时间高效暂存区的工作可追溯到 20 世纪 80 年代末 [35, 41, 42]，也与在操作系统中设计空间高效页表的问题密切相关 [1, 2, 10]。最著名的理论结果

归功于 Gonnet 和 Larson [35], 他们给出了一个仅使用 $O(n \log \epsilon^{-1})$ 比特空间的暂存区。一个推论是, 如果 $\epsilon = \Theta(1)$, 暂存区仅使用 $O(n)$ 比特。

然而, Gonnet 和 Larson 的结果有几个缺点 [35]。首先, 暂存区仅在 S 的插入/删除是随机的情况下提供可证明保证; 在 S 被任意插入/删除/查询序列修改的情况下, 设计空间高效暂存区的问题仍然开放。其次, [35] 中暂存区的内部内存操作在 RAM 模型中不是常数时间的 (甚至当 $\epsilon = o(1)$ 时也不是常数期望时间)。

通过将微型指针与构造空间高效过滤器的现代技术相结合, 我们证明可以构造一个大小的暂存区 $O(n \log \epsilon^{-1})$ 比特, 在 RAM 模型中支持常数时间操作 (不仅期望意义下, 而且以高概率), 并支持任意的插入/删除/查询序列。

2 2 预备知识

操作。 具有 q 比特值的**解引用表**是一个支持以下操作的数据结构:

- $\text{Create}(n, q)$: 该过程创建一个新的解引用表, 返回指向具有 n 个槽位 (每个大小 q 比特) 的数组的指针。我们称此数组为**存储区** (store)。
- $\text{Allocate}(k)$: 给定键 k , 该过程为 k 在存储区中分配一个槽位, 返回一个比特串 p , 我们称为**微型指针** (tiny pointer)。
- $\text{Dereference}(k, p)$: 给定键 k 和微型指针 p , 该过程返回存储区中分配给 k 的槽位的索引。如果 p 不是 k 的有效微型指针 (即 p 不是由调用 $\text{Allocate}(k)$ 返回的), 则该过程可以返回存储区中的任意索引。
- $\text{Free}(k, p)$: 给定键 k 和微型指针 p , 该过程从 k 释放槽位 $\text{Dereference}(k, p)$ 。用户仅被允许在 (k, p) 上调用此函数, 其中 p 是 k 的有效微型指针 (即 p 由最近一次调用 $\text{Allocate}(k)$ 返回)。

我们说键 k 是**存在的** (present), 如果它被分配的时间更近于被释放的时间; 在这种情况下, 最近一次调用 $\text{Allocate}(k)$ 返回的微型指针 p 被称为 k 的微型指针。用户仅被允许为每个键 k 分配最多一个微型指针 p 。即, 每次调用 $\text{Allocate}(k)$ 获得某个微型指针 p 时, 必须在再次调用 $\text{Allocate}(k)$ 之前调用 $\text{Free}(k, p)$ 。

我们说存储区中的槽位 i 是**被占用的** (occupied), 如果存在一个存在的键 k , 具有微型指针 p , 使得 $\text{Dereference}(k, p) = i$, 否则我们说它是**空闲的** (free)。我们通常将参数 n (即存储区中的槽位数) 称为表的**大小或容量**。

保证。 解引用表提供以下保证:

- 对任意两个存在的键 $k_1 \neq k_2$, 分别具有微型指针 p_1 和 p_2 , 有 $\text{Dereference}(k_1, p_1) \neq \text{Dereference}(k_2, p_2)$ 。

- $\text{Dereference}(k, p)$ 仅依赖于 k 、 p 、随机比特和参数 n 。

第二个性质确保解引用微型指针的行为类似于解引用标准指针；在两种情况下，执行解引用时不需要访问被指向的数据结构。这对于我们后面的几个应用最终很重要。特别地，它确保在外部内存应用中，每次解引用仅引发单次 I/O；并且它确保在数据结构应用中，微型指针指向的位置是稳定的（即一旦微型指针 p 分配给键 k ，被指向的位置不会改变）。

元数据信息。解引用表可以存储元数据以高效地执行更新（分配和释放）。元数据可以作为存储区的一部分存储，也可以存储在允许消耗最多 $O(n)$ 比特的辅助数据结构中。换句话说，解引用表被允许使用 $O(n)$ 比特（即每个槽位 $O(1)$ 比特的开销）的元数据作为“免费”，不计入存储区的空间消耗，但任何额外的元数据必须计入存储区的空间消耗。注意，解引用表不允许在当前已分配的存储区槽位中存储元数据。

失败概率。我们将允许分配具有小的失败概率。即，每次分配允许以 $1/\text{poly}(n)$ 的概率失败，在这种情况下分配简单地返回失败消息而非微型指针。一般来说，如果一个随机事件以概率 $1 - 1/\text{poly}(n)$ 发生，我们说它以**高概率**（with high probability，简称 w.h.p.）发生。

我们指出，在分析解引用表时，我们将始终假设分配、释放和解引用的序列由一个**健忘对手**（oblivious adversary）决定（即序列是预先确定的，而不是根据解引用表的行为自适应）。由此产生的一个后果是，如果某个分配失败，对操作序列的唯一影响是相应的释放调用被移除。

负载因子。解引用表的任何实现还必须指定一个额外参数 $\delta \in [0, 1]$ ，指示表被允许有多满。这意味着解引用表一次可以支持最多 $(1 - \delta)n$ 次分配——量 $1 - \delta$ 被称为表的**负载因子**。如果在已经有 $(1 - \delta)n$ 次分配时调用 Allocate 函数，则解引用表被允许使该分配失败。⁷

⁷ 注意，即使解引用表只保证能够一次存储最多 $(1 - \delta)n$ 次分配，我们仍然使用解引用表的术语“大小”和“容量”来指代 n ，而非 $(1 - \delta)n$ ，因为 n 表示存储区中 q 比特条目的总数。

由于解引用表可以使用最多 $O(n)$ 空间作为元数据，解引用表消耗的总空间量可能高达 $nq + O(n) = (1 - \delta)nq + \delta nq + O(n)$ 。第一项 $(1 - \delta)nq$ 是分配可以利用的空间，其他项 $\delta nq + O(n)$ 是浪费的空间。注意，考虑 $\delta \ll 1/q$ 没有意义，因为这只会使浪费的空间由元数据主导。因此，当构造具有某个负载因子 $1 - \delta$ 的解引用表时，我们始终隐含地假设 $q = \Omega(\delta^{-1})$ 。

哈希与独立性。我们的解引用表构造都将使用哈希函数。为简单起见，本文中将哈希函数视为均匀且完全独立的。这一假设不失一般性，因为已经有已知的哈希函数族 [30, 47] 以常数时间求值和 $O(n)$ 随机比特模拟 n -独立性，并且已经有充分理解的技术 [8, 43] 将这些族应用于需要 $\text{poly}(n)$ -独立性的数据结构。⁸ 这些已知技术可以轻松地直接应用于我们所有的数据结构；唯一的注意点是所使用的哈希函数族 [30, 47] 给数据结

构引入了其自身的额外 $1/\text{poly}(n)$ 失败概率。因此，即使一个数据结构在完全随机哈希函数的假设下提供亚多项式失败概率，如果我们希望使用显式的哈希函数族，那么我们必须允许 $1/\text{poly}(n)$ 的失败概率。

⁸ 基本思想是简单地将容量为 n 的数据结构替换为 $n^{1-\epsilon}$ 个容量为 n^ϵ 的数据结构。完整数据结构中的每个元素 x 被随机哈希到 $n^{1-\epsilon}$ 个数据结构之一，每个数据结构只需要 $\text{poly}(n^\epsilon) = o(n)$ 的独立性。

3 3 固定大小指针的上界

在本节中，我们给出固定大小微型指针的最优构造。我们证明以下定理：

定理 1. 对于每个 $\delta \in (0, 1)$ ，存在一个解引用表，(i) 每次分配以高概率成功，(ii) 负载因子至少为 $1 - \delta$ ，(iii) 以高概率具有常数时间更新，(iv) 微型指针大小为 $O(\log \log \log n + \log \delta^{-1})$ 。

特别地，对于 $\delta = 1/\log \log n$ ，我们得到大小为 $O(\log \log \log n)$ 的微型指针。因此，我们可以双重指数地击败原始的 $\log n$ 比特指针，同时仍支持 $1 - o(1)$ 的负载因子。

该证明是我们微型指针构造中最简单的，使用了两个算法构建块。

第一个构建块：负载均衡表。 负载均衡表 (load-balancing table) 是一种具有非常特定内部表示的简单解引用表类型，并且与普通解引用表不同，它被允许以不可忽略的概率在 Allocate 调用上失败。粗略地说，如果负载均衡表具有负载因子 $1 - \delta$ ，则负载均衡表被允许在 δ 比例的分配上失败。

负载均衡表实现如下。如果存储区大小为 m ，我们将其划分为 m/b 个大小为 $b = \Theta(\delta^{-2} \log \delta^{-1})$ 的桶。为分配键 k ，我们使用哈希函数 h 将 k 哈希到其中一个桶。如果桶 $h(k)$ 包含一个空闲槽位，则我们在该桶内分配任意空闲槽位 $i \in [b]$ ，并返回 i 作为微型指针。否则，桶中所有 b 个槽位都被占用，分配失败。函数 Dereference(k, p) 可以实现为简单返回桶 $h(k)$ 中的第 p 个槽位。

负载均衡表将作为我们构造的解引用表中的构建块。基本思想是，我们可以使用负载均衡表处理除 δ 比例之外的所有分配，剩余分配可以通过某种其他机制处理。因此，我们将需要以下引理，该引理给出在任何给定时刻失败分配总数的界：

引理 1. 考虑一个大小为 m 、负载因子为 $1 - \delta$ 的负载均衡表。考虑一系列分配和释放，使得在任何给定时刻最多进行 $(1 - \delta)m$ 次分配。如果一次分配失败，并且该分配本应在某个时间 t 被释放，则我们认为该分配在该时间 t 之前是**存活的** (alive)。在任何给定时刻，已失败且仍然存活的分配数量为 $O(\delta m)$ ，概率至少为 $1 - \exp(-\text{poly}(\delta)m)$ 。

我们指出，在引理 1 的所有应用中，我们将不失一般性地有 $\log \delta^{-1} = o(\log m)$ （否则，我们将有 $\log \delta^{-1} = \Omega(\log m)$ ，因此可以直接使用标准的 $O(\log m)$ 比特指针）。因此引理提供的概率界将始终至少为 $1 - \exp(m^{1-o(1)}) \geq 1 - 1/\text{poly}(m)$ 。

我们将引理 1 的证明推迟到第 7.1 节，该节建立了该引理的一个更一般版本。尽管

证明是非平凡的，因为同一键可能被多次分配/释放/重新分配导致相互依赖性，但我们不将其视为本文的主要技术贡献之一。这是因为引理 1 轻松地源于当前作者在另一篇关于空间高效哈希表的近期论文 [11] 中建立的一个引理。尽管如此，我们在第 7.1 节中给出了一个替代证明，既是为了完整性，也因为该证明采用了与我们过去工作有些不同（且更优雅）的方法，并且为了涵盖更大的参数范围。

为总结我们对负载均衡表的讨论，我们必须描述如何以常数时间实现分配和释放。这里有两种情况，取决于 b 与负载均衡表所用解引用表大小 n 的比较。

如果 $b \leq \log n$ ，则我们可以为每个桶存储一个 b 比特的位图，指示桶中哪些槽位是空闲的；我们可以使用标准位操作在位图上以常数时间实现分配和释放函数。

如果 $b > \log n$ ，我们采用不同的方法。在这种情况下，我们声称不失一般性地， $q = \Omega(\log b)$ ，其中 q 是被存储元素的比特大小（我们将马上证明此论断）。此论断意味着我们可以如下跟踪负载均衡表每个桶中哪些槽位空闲：我们简单地在每个桶中存储一个**空闲列表**（free list），即由所有空闲槽位组成的链表，其中每个空闲槽位包含指向链表中下一个空闲槽位的指针。这是可能的，因为每个空闲槽位为 q 比特，而链表中的每个指针仅需要 $\log b = o(q)$ 比特。 m/b 个链表的 $\log b$ 比特基指针可以存储在大小为 $O((m/b) \cdot \log b) \subseteq O(m)$ 的辅助元数据数组中，其中 m 是负载均衡表的大小。空闲列表使我们能够以常数时间实现分配和释放函数。

为证明此空闲列表方法有效，尚需证明不失一般性地 $q = \Omega(\log b)$ 。令 $1 - \delta$ 为完整解引用表（负载均衡表是其中一部分）的负载因子。令 $1 - \tilde{\delta}$ 为负载均衡表的负载因子。由于 $b > \log n$ ，我们必须有 $\tilde{\delta}^{-1} = \Omega(\sqrt{\log n})$ （否则 $\tilde{\delta}^{-1} = O(\sqrt{\log n})$ ，因此 $b = O(\log n)$ ）。在我们所有的构造中，如果我们使用负载因子 $1 - \tilde{\delta}$ 满足 $\tilde{\delta}^{-1} = \tilde{\Omega}(\sqrt{\log n})$ 的负载均衡表，我们将始终有 $\log \tilde{\delta}^{-1} = \Theta(\log \delta^{-1})$ 。回忆，如果解引用表具有负载因子 $1 - \delta$ ，则假设解引用表存储大小为 $q = \Omega(\delta^{-1})$ 比特的对象。因此，我们有 $q = \Omega(\log \delta^{-1}) = \Omega(\log \tilde{\delta}^{-1}) = \Omega(\log b)$ ，正如所需。

第二个构建块：二次幂选择解引用表。为补偿负载均衡表的高失败概率，我们开发了第二个构建块：一个简单的解引用表，支持 $O(\log \log \log n)$ 比特微型指针，并且与负载均衡表不同，具有低失败概率。此构建块的缺点是它只支持非常小的负载因子。

引理 2。存在一个 δ 满足 $1 - \delta = \Theta(1/\log \log n)$ ，使得存在一个解引用表，(i) 每次分配以高概率成功，(ii) 负载因子至少为 $1 - \delta$ ，(iii) 以高概率具有常数时间更新，(iv) 微型指针大小为 $O(\log \log \log n)$ 。

证明。我们将存储区划分为大小为 $b = \Theta(\log \log n)$ 的桶。当调用 $\text{Allocate}(k)$ 时，键 k 被哈希到两个桶 $h_1(k), h_2(k) \in [1, n/b]$ 。键 k 被分配到两个桶中包含最多空闲槽位的那个桶中的一个槽位。微型指针 p 为 $1 + \log b = O(\log \log \log n)$ 比特，指示在两个桶中哪个槽位被分配。

我们可以将分配视为使用二次幂选择（power-of-two-choices）规则 [55, 56] 插入桶中的球，同一个球可能随时间被插入/删除/重新插入。由于负载因子为 $\Theta(1/\log \log n)$ ，每个

桶中球的期望数量为 $O(1)$ 。在此设置中, 已知以高概率, 最满桶中的球数为 $O(\log \log n)$ [56]。因此分配以高概率成功。

最后, 为以常数时间实现分配和释放, 我们可以仅使用位图跟踪每个桶中哪些槽位空闲; 由于每个桶仅 $O(\log \log n)$ 个槽位, 位图每个仅 $O(\log \log n)$ 比特, 因此每个位图适合一个机器字。使用标准位操作, 位图可用于以每次分配/释放常数时间跟踪哪些槽位空闲 (并在常数时间内为给定分配找到空闲槽位)。位图总共消耗 $O(n)$ 比特空间。□

组合各部分。当然, 二次幂选择解引用表本身不是很有用, 因为它们只支持 $o(1)$ 的负载因子。我们现在展示如何将它们与负载均衡表结合以证明定理 1。

图 1: 主表和次级表的布局图示。主表实现为支持负载因子 $1 - \frac{\delta}{2}$, 使得一次只有 $\frac{\delta}{2}n$ 个分配溢出到次级表。次级表实现为具有大小 $n' = n/2$ 并支持 (更稀疏的) 负载因子 $\Theta(1/\log \log n') = \Theta(\delta)$, 使得它可以成功存储所有来自主表的溢出分配。

定理 1 的证明。由于我们愿意拥有大小为 $\Theta(\log \log \log n + \log \delta^{-1})$ 的微型指针, 我们可以不失一般性地假设 $\delta = o\left(\frac{1}{\log \log n}\right)$ 。

我们将 $1 - \frac{\delta}{2}$ 比例的分配存储在大小为 $m = (1 - \delta/2)n$ 个槽位的负载均衡表中, 该表支持负载因子 $1 - \delta/(2c)$ (c 为某个足够大的正常数); 我们称之为主表 (primary table)。在主表中失败的分配被存储在次级表 (secondary table) 中, 该表使用引理 2 实现, 具有 $n' := n/2$ 个槽位并支持负载因子 $1 - \delta' := \Theta(1/\log \log n')$ 。如果一次分配在次级表中失败, 或者次级表的负载因子超过 $\Theta(1/\log \log n')$, 则该分配在整个解引用表中也失败。注意主表和次级表的总大小 (以槽位计) 为 n 。图 1 给出了两个表布局的图示。

由于主表和次级表都是常数时间的, 完整解引用表也是常数时间的。此外, 每次分配返回一个微型指针, 该指针可以指向主表或次级表 (加上 1 比特信息指示它指向哪个表)。由于主表和次级表都具有大小为 $O(\log \log \log n + \log \delta^{-1})$ 的微型指针, 关于微型指针大小的论断也得到证明。

我们最后的任务是界定给定分配失败的概率。引理 1 告诉我们, 以高概率, 在任何给定时刻次级表中的分配数最多为 $\frac{\delta}{2}n$ 。由于次级表具有 $n' = \Theta(n/2)$ 个槽位, 且由于 $\delta = o\left(\frac{1}{\log \log n}\right)$, 因此以高概率在任何给定时刻次级表中的分配数为 $o(n'/\log \log n) = o(n'/\log \log n')$ 。因此我们从引理 2 得出, 次级表中的分配每个关于 n' 以高概率成功。不失一般性地, $n' \geq \sqrt{n}$ (否则 $\delta = O(1/\sqrt{n})$, 我们可以直接使用标准 $\log n$ 比特指针)。因此次级表中的分配每个关于 n 以高概率成功。□

4 4 可变大小指针的上界

在本节中, 我们给出可变大小微型指针的最优构造。我们证明以下定理:

定理 2。对于每个 $\delta \in (0, 1)$, 存在一个解引用表, (i) 每次分配以高概率成功, (ii) 负载因子至少为 $1 - \delta$, (iii) 以高概率具有常数时间更新, (iv) 微型指针大小为 $O(P + \log \delta^{-1})$, 其中 P 是一个随机变量, 满足 $\Pr[P \geq i] \leq 2^{-2^{\Omega(i)}}$ 对所有 i 成立。特别地, 微型指针大

小的期望为 $O(1 + \log \delta^{-1})$ 。

我们可以不失一般性地假设 $1 - \delta < \zeta$ ，其中 ζ 为我们选择的某个足够小的正常数（如果 $1 - \delta > \zeta$ ，我们可以重置 $\delta = 1 - \zeta = \Theta(1)$ 而不改变定理的保证）。

注意到，使用与定理 1 证明中相同的主/次级表构造，我们可以立即归约到负载因子为我们选择的一个正常数的情况。事实上，假设我们可以实现一个解引用表 T' ，具有某个正常数 $\zeta > 0$ 的负载因子 ζ 和 $O(1)$ 的平均微型指针大小。那么我们可以使用 T' 作为构造中的次级表：如果整个解引用表支持负载因子 $1 - \delta$ ，则对次级表的要求是它必须能够使用 $n/2$ 个槽位支持 $\frac{\delta}{2}n$ 个元素。因此只要 $\delta < \zeta/2$ （这总是不失一般性的）， T' 就是足够的。

因此，我们证明定理 2 的任务归约为证明以下命题。

命题 1。 存在一个解引用表，(i) 以高概率成功，(ii) 负载因子为 $\Theta(1)$ ，(iii) 以关于 n 的高概率具有常数时间更新，(iv) 微型指针大小为 P ，其中 P 是满足对任意 i 有 $\Pr[P \geq i] \leq 2^{-2^{\Omega(i)}}$ 的随机变量。

为便于讨论，在本节的其余部分，我们用 n 表示解引用表中最多可存储的项数（而不是槽位数），我们的目标是构造一个具有 $O(n)$ 个槽位的解引用表。

构造解引用表。 我们现在描述用于证明命题 1 的解引用表构造。解引用表最初将每个键哈希到 $n/\log n$ 个容器（container）之一，使得在任何时刻，每个容器期望有 $\Theta(\log n)$ 个项。我们确定性地将每个容器中的元素数限制为 $s = c \log n$ 个项，其中 $c > 1$ 为某个待定的足够大的常数。当一个键被哈希到一个已有 $c \log n$ 个项的容器时，分配失败。

每个容器独立管理，其分配/释放使用具有 $\log_2 s$ 层的方案执行，如下所述。对于每个 $0 \leq i < \log_2 s$ ，第 i 层是一个负载均衡表，具有 $s_i := s/2^i$ 个桶，每个桶 b 个槽位，其中 $b \geq 2$ 为某个待定的足够大的常数。

基本思想是，当第 i 层的一次分配因桶满而失败时，我们递归地在下一层 $i+1$ 尝试该分配（使用与第 i 层不同的哈希函数）。直观上，只要 b 是足够大的常数，每层应在其至少 $1/2$ 的分配上成功，这就是为什么下一层 $i+1$ 可以是前一层一半大小的原因。

此基本构造的问题是，如果仅连续几层表现糟糕，导致 $\Omega(s_i)$ 个元素被发送到某层 i ，那么在层 $i, \dots, \log_2 s$ 的组合中可能没有空间容纳这些元素。另一方面，我们的构造必须能够处理这种糟糕的情况，因为大多数层太小，我们无法对其行为提供高概率保证。因此，我们必须修改构造，使得当某层表现糟糕时，其影响被隔离。

为此，我们为每层添加一个回退结构，称为**溢出数组**（overflow array），以防止过度占用。第 i 层的溢出数组具有 s_i 个槽位（与该层负载均衡表相同的槽位数）。令 L_i 为表示当前存储在层 i 或更高层（包括其溢出数组）中值数量的随机变量。每当某层 i 的分配失败（因桶满），我们仅在 $L_{i+1} < s_{i+1}$ 时递归地在下一层分配，否则，我们将值放入第 i 层溢出数组中的任意可用槽位。这样做的结果是，我们确定性地保证对每个层 i （包括 $i = 0$ ，对此这平凡成立，因为 $s_0 = s$ ）都有 $L_i \leq s_i$ 。

重要的是，没有溢出数组会耗尽空间：由于 $L_i \leq s_i$ （确定性地），第 i 层溢出数组中的元素总数也确定性地最多为 s_i ，这正是溢出数组的容量。

我们现在可以描述完整的分配算法了。图 2 给出了实现每个容器所用布局的图示。

图 2：用于实现每个大小为 $\Theta(\log n)$ 的容器的布局图示。当一次分配在第 i 个负载均衡表中失败时，它要么进入第 $(i+1)$ 个负载均衡表（如果 $L_{i+1} < s_{i+1}$ ），要么进入第 i 个溢出数组（确定性地保证有空闲槽位）。

算法 Allocate(k):

- (1) 将 k 哈希到 $n/\log n$ 个容器之一。
- (2) 如果选中的容器已经达到满容量 s ，则失败。
- (3) 否则，在选中的容器中分配 k ：
 - (a) 对每个 $i = 0, 1, \dots, \log_2 s - 1$ ：
 - i. 递增 L_i 。
 - ii. 尝试在第 i 个负载均衡表中分配 k 。
 - iii. 如果分配成功：
 - 令 j 为所选桶中选中的槽位。
 - 返回 (层 i , 负载均衡表, 桶槽位 j)。
 - iv. 如果 $L_{i+1} \geq s_{i+1}$ ：
 - 在第 i 个溢出数组中选取任意空闲槽位。
 - 令 j 为该数组中所选的槽位。
 - 返回 (从后面数的层 $\log_2 s - 1 - i$, 溢出数组, 槽位 j)。

注意，如果一次分配最终使用第 i 层负载均衡表中某桶的槽位 j ，则微型指针编码：量 i ($O(\log i)$ 比特)；分配使用负载均衡表而非溢出数组这一事实 ($O(1)$ 比特)；以及量 j ($O(\log b) = O(1)$ 比特)。这种情况下微型指针的总长度为 $O(\log i)$ 。⁹

⁹ 我们遵循约定，对所有 i 有 $\log i = \Theta(1)$ ，所以 $\log 0$ 和 $\log 1$ 设为 1。

另一方面，如果一次分配最终使用第 i 层溢出数组中的第 j 个槽位，则微型指针编码：量 $\log_2 s - 1 - i$ ($O(\log(\log_2 s - 1 - i))$ 比特)；分配使用溢出数组而非负载均衡表这一事实 ($O(1)$ 比特)；以及量 j ($O(\log s_i)$ 比特)。重要的是，在这种情况下，我们选择编码 $\log_2 s - 1 - i$ ，而非等价的量 i 。这使我们能够将微型指针的总大小界定为

$$O(\log(\log_2 s - 1 - i)) + O(1) + O(\log s_i) = O(\log \log(s/2^i) + \log s_i) = O(\log \log s_i + \log s_i) = O(\log s_i).$$

因此，当一次分配使用第 i 层的溢出数组时，我们可以将微型指针大小界定为 $O(\log s_i)$ 。

以常数时间实现操作。微型指针中的信息使解引用能够轻松地在 $O(1)$ 时间内执行。然而，以 $O(1)$ 时间执行分配和释放稍微困难一些。

让我们从考虑实现分配的朴素方法开始，看看为什么这太慢。我们必须首先确定使用哪个容器（这只需要求值一个哈希函数，花费常数时间）。然后我们必须确定我们将使用哪一层；如果我们最终使用第 i 层，则需要时间 $\Theta(i)$ ，当 $i = \omega(1)$ 时这太慢。

我们如下解决此问题。令 d 为某个足够大的正常数。我们将使用朴素方法实现层 $0, 1, \dots, d-1$ ，然后我们将使用**四俄罗斯方法**（Method of Four Russians，即“查找表方法”）实现层 $d, \dots, \log_2 s$ 。注意层 $d, \dots, \log_2 s$ 中的总槽位数最多为 $4s_d/2^d \leq (\log n)/10$ 。因此这些层中哪些槽位空闲的整个状态可以在 $(\log n)/10$ 比特中编码；我们将此量作为每个容器的元数据存储，总计在所有 $n/\log n$ 个容器中为 $O(n)$ 比特元数据。此外，用于在每层中选择桶的哈希 $h_1(k), h_2(k), \dots, h_{\log_2 s}(k)$ 总共仅表示 $O((\log \log n)^2)$ 比特（并且可以实现为仅是一个哈希函数的前 $O((\log \log n)^2)$ 比特）。因此，层 $d, \dots, \log_2 s$ 的整个状态，加上关于哈希 $h_1(k), h_2(k), \dots, h_{\log_2 s}(k)$ 的所有信息，可以在一个整数 σ 中编码为 $O((\log \log n)^2) = o(\log n)$ 比特。这意味着我们可以预先构造一个大小为 $n^{o(1)}$ 的查找表，该表可用于对任意给定的 σ 值确定分配应该使用哪一层。该查找表占用可忽略的 $\tilde{O}(\sqrt{n})$ 元数据空间量，允许在 $O(1)$ 时间内执行分配，并且可以在解引用表创建时构造。

既然我们已经指定了如何实现分配，释放就容易实现了，因为它们只需更新元数据以反映槽位已被释放（这只需翻转元数据中的单个比特）。

我们现在已经完全指定了解引用表的构造和实现。尚需分析其性质，即失败概率、负载因子和微型指针大小。

失败概率。分配可能失败的唯一方式是，它哈希到的容器中没有空间，即该容器已经有 $c \log n$ 个元素。否则，如果容器中少于 $c \log n$ 个元素，则分配保证成功（但当然，不保证产生小的微型指针）。

平均而言， $\Theta(\log n)$ 个项哈希到任何特定容器，因此由 Chernoff 界，对于某个正常数 c ，以关于 n 的高概率，所有容器中的最大大小最多为 $c \log n$ 。由联合界，这对所有 $n/\log n$ 个容器同时以关于 n 的高概率成立。因此，如果我们选择 $s = c \log n$ 对某个足够大的常数 c ，在任何时间点，所有容器都将以关于 n 的高概率低于容量。

负载因子。接下来，我们验证总槽位数为 $O(n)$ 。每个容器的解引用表使用空间 $O(\sum_i s_i) = O(s_0) = O(s) = O(\log n)$ 个槽位，且有 $n/\log n$ 个容器。因此，总空间为 $O(n)$ ，所以负载因子为 $\Theta(1)$ ，正如所需。

微型指针大小。为完成命题 1 的证明，我们分析在给定分配不失败的条件下，该分配的微型指针大小。微型指针的大小取决于键最终被分配到哪里。具体来说，它是：

- $O(\log i)$, 如果键被分配在第 i 个负载均衡表中;
- $O(\log s_i)$, 如果键被分配在第 i 个溢出数组中。

固定一个任意容器作为分配发生的位置, 考虑以下事件:

- B_i : 键被分配在第 i 个负载均衡表中;
- O_i : 键被分配在第 i 个溢出数组中;
- $\mathcal{L}_i$: $L_i < s_i$ 。

我们将条件于两个事件: (i) 该元素选择了我们固定的容器, (ii) 该容器包含少于 $c \log n$ 个元素 (即分配不失败)。为清晰起见, 我们将省略条件符号。令 P 为输出微型指针的大小。那么, 由条件期望律,

$$\mathbb{E}[P] \leq \sum_i \Pr[B_i] \cdot O(\log i) + \sum_i \Pr[O_i] \cdot O(\log s_i).$$

我们分别界定每一项。一方面,

$$\Pr[B_i] \leq \Pr[B_0, \mathcal{L}_1, B_1, \dots, \mathcal{L}_{i-1}, B_{i-1}]$$

$$\leq \Pr[B_0] \cdot \Pr[B_1 \mid B_0, \mathcal{L}_1] \cdots \Pr[B_{i-1} \mid B_0, \mathcal{L}_1, \dots, B_{i-2}, \mathcal{L}_{i-1}].$$

对每个 j , 第 j 层的负载因子最多为 $1/b$, 因为存在 $L_j < s_j$ 个项, s_j 个桶, 且每个桶容量为 b 。这意味着确定性地最多 $1/b$ 的桶是满的, 因此选中满桶的概率最多为 $1/b$ 。因此, 方程 (2) 中的每一项都有界 $1/b$, 且

$$\Pr[B_i] \leq 1/b^i \leq 1/2^i.$$

另一方面,

$$\Pr[O_i] \leq \Pr[\overline{\mathcal{L}_{i+1}}].$$

我们可以使用引理 1 界定后一个概率。由构造, 第 i 层的负载均衡表始终有最多 s_i 次分配对之进行 (包括失败的那些, 因为 $L_i \leq s_i$ 且 L_i 同时计数第 i 层中的元素和第 $i+1, i+2, \dots$ 层中的元素); 此外, 在表上执行的分配和释放独立于表中使用的随机性。假设桶大小 b 是足够大的常数, 因此我们可以应用引理 1 推导出, 以至少

$$1 - \exp(-\text{poly}(b)s_i) = 1 - \exp(-\Omega(s_i))$$

的概率，在任何给定时刻第 i 层失败分配的数量少于 $s_i/2 = s_{i+1}$ （因此 $\mathcal{L}_{i+1}$ 成立）。因此，我们可以得出结论

$$\Pr[O_i] \leq 1/2^{\Omega(s_i)}.$$

组合各部分：

$$\mathbb{E}[P] = \sum_i \frac{O(\log i)}{2^i} + \sum_i \frac{O(\log s_i)}{2^{\Omega(s_i)}} = O(1).$$

注意这些计算表明，大小为 $O(\log \ell)$ 的微型指针具有概率 $2^{-\Omega(\ell)}$ ，或者等价地，大小为 $O(\ell)$ 的微型指针具有概率 $2^{-2^{\Omega(\ell)}}$ 。这提示微型指针大小以双重指数速率衰减。我们接下来证明这一点。对任意 ℓ ，

$$\begin{aligned} \Pr[P \geq \ell] &\leq \sum_{i: O(\log i) \geq \ell} \Pr[B_i] + \sum_{i: O(\log s_i) \geq \ell} \Pr[O_i] \\ &= \sum_{i \geq 2^{\Omega(\ell)}} \Pr[B_i] + \sum_{s_i \geq 2^{\Omega(\ell)}} \Pr[O_i] \\ &\leq \sum_{i \geq 2^{\Omega(\ell)}} \frac{1}{2^i} + \sum_{s_i \geq 2^{\Omega(\ell)}} \frac{1}{2^{\Omega(s_i)}}. \end{aligned}$$

两个和都由其首项主导，因此为 $1/2^{2^{\Omega(\ell)}}$ 。因此，

$$\Pr[P \geq \ell] \leq \frac{1}{2^{2^{\Omega(\ell)}}},$$

这完成了命题 1 的证明。如前所述，命题 1 反过来蕴含定理 2。

界定微型指针大小的和。在我们的微型指针应用中，使用可变大小指针的一种常见方式是将 $\Theta\left(\frac{\log n}{\log \delta^{-1}}\right)$ 个微型指针打包到 $O(\log n)$ 比特中。因此，我们通过证明一组 S 的 $O(\log n / \log \delta^{-1})$ 个微型指针消耗的总比特数的界来结束本节。

命题 2。使用定理 2 中的构造，对任意集合 S 的 $O\left(\frac{\log n}{\log \delta^{-1}}\right)$ 个微型指针，它们大小之和以高概率为 $O(\log n)$ 比特。

证明。以高概率， S 的所有分配都成功。这意味着我们可以忽略分配失败的情况，因此当分配失败时，我们将其视为贡献大小为 0 的微型指针。令 K 为对应于 S 中微型指针的键集合。简单的情况是，如果每个键 $k \in S$ 哈希到不同的容器；在这种情况下，我们可以独立分析每个容器，得出每个微型指针 $\text{Allocate}(k)$ 独立地具有长度 $O(\log \delta^{-1} + P_k)$ 比特，其中 $\Pr[P_k > \ell] \leq 2^{-2^{\Omega(\ell)}}$ 。对独立几何随机变量和施加 Chernoff 界，我们可以得出结论，以高概率 $\sum_{k \in K} P_k = O(\log n)$ ，因此 S 消耗的总比特数为 $O(\log n)$ 。

如果 K 中的某些键 k 与 K 中的其他键 k' 哈希到同一容器怎么办？那么我们就不能再独立地分析所得微型指针的长度。令 X 表示这样的键的集合 k 。由于每个微型指针确定性地最多 $O(\log n)$ 比特，我们可以通过证明以高概率 $|X| = O(1)$ 来完成证明。

令 $k_1, k_2, \dots$ 表示 K 中的键, 令 X_i 为事件“ k_i 哈希到与 $k_1, k_2, \dots, k_{i-1}$ 之一相同的容器”的指示随机变量。则 $|X| \leq 2 \sum_i X_i$ 。另一方面, 每个 X_i 独立地满足 $\Pr[X_i] \leq (i-1)/n \leq |S|/n = O(\log n/n)$ 。因此 $\sum_i X_i$ 是独立指示随机变量之和, 总均值为 $O(\log^2 n/n)$ 。施加 Chernoff 界, 我们得出结论以高概率 $\sum_i X_i = O(1)$, 这完成了证明。 $\square$

5 5 下界

在本节中, 我们证明定理 1 和 2 中的界是紧的。我们首先证明可变大小微型指针的下界, 因为它随后被用作固定大小情况证明的一部分。

使得可变大小微型指针的下界棘手的是, 任何单个微型指针可能非常小。例如, 解引用表可以有一个 (对每个键) 对应于微型指针 0 的单个特殊槽位, 然后如果解引用表想要使某个微型指针变小, 它可以分配该特殊槽位。因此, 我们的证明对不同类型的槽位区别对待: 对每个槽位 j , 我们定义一个势函数 $\phi(j)$, 指示该槽位对随机插入有多“有用”。思想是, 使用具有小势 $\phi(j)$ 的槽位 j 的插入, 平均而言必须具有相对大的微型指针; 但使用具有大势 $\phi(j)$ 的槽位 j 的插入必须是稀少的, 因为只有相对小比例的槽位可以具有大势, 并且对它们的插入次数可以由对它们的删除次数来界定。

定理 3. 考虑键的全空间 U , 假设 U 具有足够大的多项式大小。如果解引用表支持期望大小为 s 的可变大小微型指针和负载因子 $1 - \delta = \Theta(1)$, 则 $s = \Omega(\log \delta^{-1})$ 。

证明。 令 U 为大小为 n^c 的全空间, 其中 c 为足够大的常数。令 $\delta < 1/4$ 。令 T 为具有 n 个槽位和负载因子 $1 - \delta$ 的解引用表 (即它能够一次从 U 中的键分配最多 $(1 - \delta)n$ 个槽位)。此外, 假设 T 保证期望微型指针长度最多为 $\bar{\ell}$ 。那么我们希望证明

$$\bar{\ell} = \Omega(\log \delta^{-1}).$$

为简化讨论, 我们将键 $k \in U$ 视为驻留在分配给它的位置中。因此分配对应于插入, 释放对应于删除。

考虑一个工作负载, 其中表被初始化为包含 $(1 - \delta)n$ 个任意元素, 然后我们在插入和删除之间交替, 进行 $n^{c/2}$ 步。每次插入选择一个随机的 U 中元素 (以关于 n 的高概率, 我们从不插入一个已经存在的元素), 每次删除从存在的元素中随机选择一个。

我们将微型指针视为在 $\mathbb{N}$ 中取值。如果微型指针取值为 i , 则它使用 $\Theta(\log i)$ 比特。对每个元素 $x \in U$, 令 $h_i(x)$ 表示如果 x 有值为 i 的微型指针, x 在 T 中将驻留的位置。设 $\alpha = \delta^{-1}/32$ 。对表中每个位置 $j \in [n]$, 定义势函数 $\phi(j)$ 为

$$\phi(j) = \frac{|\{u \in U, i \in [\alpha] \mid h_i(u) = j\}|}{|U|}.$$

称一次插入是**安全的** (safe), 如果插入的元素 x 被插入到 $h_1(x), \dots, h_\alpha(x)$ 的位置

之一。称一次插入是**资源效率的** (resource efficient), 如果插入的元素 x 被插入到满足 $\phi(j) \leq \frac{4\alpha}{n}$ 的位置 j 。

一次给定插入同时安全和资源效率的概率最多为

$$\begin{aligned}
& \sum_{\text{空闲位置 } j \in [n]} \sum_{i=1}^{\alpha} \Pr[h_i(x) = j] \cdot \mathbf{1}_{\phi(j) \leq \frac{4\alpha}{n}} \\
&= \sum_{\text{空闲位置 } j \in [n]} \sum_{i=1}^{\alpha} \frac{1}{|U|} \sum_{x \in U} \mathbf{1}_{h_i(x)=j} \cdot \mathbf{1}_{\phi(j) \leq \frac{4\alpha}{n}} \\
&= \sum_{\text{空闲位置 } j \in [n]} \phi(j) \cdot \mathbf{1}_{\phi(j) \leq \frac{4\alpha}{n}} \\
&\leq \frac{4\alpha}{n} \sum_{\text{空闲位置 } j \in [n]} 1 \\
&\leq n \cdot \frac{4\alpha}{n} \\
&= 4\alpha \\
&= \frac{1}{8}.
\end{aligned}$$

由此得出, 预期为安全且资源效率的插入次数最多为 $n^{c/2}/8$ 。

接下来我们界定安全但不资源效率的插入的期望次数 A 。我们不直接界定 A , 而是检查被删除元素从满足 $\phi(j) > \frac{4\alpha}{n}$ 的位置 j 中被删除的删除次数 B 。特别注意到,

$$A \leq B + n.$$

由 $\phi(j)$ 的定义, 我们有 $\sum_{j=1}^n \phi(j) = \alpha$ 。由此得出 $|\{j \in [n] \mid \phi(j) > \frac{4\alpha}{n}\}| \leq n/8$ 。因此每次随机删除从满足 $\phi(j) > \frac{4\alpha}{n}$ 的位置 j 中移除元素的概率最多为 $\frac{n/8}{(1-\delta)n} \leq 1/4$ 。因此 $\mathbb{E}[B] \leq n^{c/2}/4$, 这意味着

$$\mathbb{E}[A] \leq n^{c/2}/4 + n \leq n^{c/2}/2.$$

由于期望为安全且资源效率的插入次数最多为 $n^{c/2}/8$, 且期望为安全且资源不效率的插入次数最多为 $n^{c/2}/2$, 期望为安全的插入次数最多为 $\frac{5}{8}n^{c/2}$ 。因此期望为不安全的插入次数至少为 $\frac{3}{8}n^{c/2}$ 。每次不安全插入产生长度至少为 $\Omega(\log \alpha) = \Omega(\log \delta^{-1})$ 比特的微型指针。由于常数比例的插入预期产生长度至少为 $\Omega(\log \delta^{-1})$ 的微型指针, 我们必须有 $\bar{\ell} = \Omega(\log \delta^{-1})$ 。□

接下来我们证明固定大小微型指针的下界, 这表明定理 1 中的界是紧的。

定理 4. 考虑键的全空间 U , 假设 U 具有足够大的多项式大小。如果解引用表支持大小为 s 的固定大小微型指针和负载因子 $1 - \delta = \Theta(1)$, 则 $s = \Omega(\log \log \log n + \log \delta^{-1})$ 。

只需证明 $s = \Omega(\log \log \log n)$ 即可, 因为我们已经证明了 $s = \Omega(\log \delta^{-1})$ 。

该证明重新利用了一个经典的球与桶下界。称一个放球规则是**顺序的** (sequential), 如果球被顺序放置, 不知道未来球的到达, 并且球在被放置后从不动。

定理 5 ([55] 中的定理 2)。 假设 m 个球使用任意顺序放球规则被顺序放入 m 个桶中, 该规则根据 $[m]^d$ 上的任意概率分布为每个球随机选择 d 个桶。则以高概率, 最满桶中的球数为 $\Omega((\log \log m)/d)$ 。

现在我们证明定理 4。

定理 4 的证明。 反证。假设存在一个负载因子为 $1 - \delta = \Theta(1)$ 且支持大小为 $s = o(\log \log \log n)$ 比特的固定大小微型指针的解引用表。令 n 为解引用表中的槽位数, 令 $m = (1 - \delta)n$ 为解引用表一次能支持的最大分配数; 不失一般性假设 $1/(1 - \delta) \in \mathbb{N}$, 因此 n 是 m 的倍数。最后, 令 $S = 2^s$, 注意到由假设 $S = o(\log \log n)$ ——且由于 $m = \Theta(n)$, $S = o(\log \log m)$ 。

回忆 U 是键取自的全空间。对每个键 $x \in U$, 定义序列 $h_1(x), h_2(x), \dots, h_S(x) \in [m]$, 使得 $h_i(x) = \lceil \frac{m}{n} \text{Dereference}(x, i) \rceil$ 。注意, 由 Dereference 函数的定义, 序列 $h_1(x), h_2(x), \dots, h_S(x)$ 仅是 x, i, n 和解引用表随机比特的函数——因此, 该序列由硬币抛掷预先确定, 且独立于所执行的分配/释放序列。令 $R \in [m]^S$ 为通过随机选择 $x \in U$ 并设 $R = (h_1(x), h_2(x), \dots, h_S(x))$ 获得的随机变量; 令 $\mathcal{R}$ 为 R 的概率分布。

我们现在将构造一个将 m 个球映射到 m 个桶的顺序放球规则。我们的规则独立地为每个球分配一个随机的从 $\mathcal{R}$ 中抽取的 S 个桶的桶序列 $h_1, h_2, \dots, h_S$ 。等价地, 我们可以将 m 个球视为 m 个键 $x_1, x_2, \dots, x_m$, 其中每个 x_i 从 U 中均匀独立随机选取, 且每个 x_i 有桶序列 $h_1(x), h_2(x), \dots, h_S(x) \in [m]^S$ 。

由于 $|U| \gg \text{poly}(n)$, 以关于 n 的高概率, x_i 们互不相同。我们的放球规则使用我们的解引用表决定将球放在哪里。为将球 x_i 放入桶中, 我们计算 $p_i = \text{Allocate}(x_i)$, 并将 x_i 放入 x_i 的桶序列中的第 p_i 个桶, 该桶由下式给出:

$$h_{p_i}(x_i) = \left\lceil \frac{m}{n} \text{Dereference}(x_i, p_i) \right\rceil \in [m].$$

总而言之, 我们构造了一个顺序放球规则, 将 m 个球顺序放入 m 个桶中, 并根据 $[m]^d$ 上的概率分布 $\mathcal{R}$ 为每个球选择 $d = S$ 个桶。由定理 5, 我们可以推导出以关于 m 的高概率, 最满桶包含至少

$$\Omega((\log \log m)/d) = \Omega((\log \log m)/S) = \omega(1)$$

个球。

另一方面, 解引用表保证对每个 $i \in [m]$, $\text{Dereference}(x_i, p_i) \in [n]$ 是唯一的。因此对给定 j 满足

$$\left\lceil \frac{m}{n} \text{Dereference}(x_i, p_i) \right\rceil = j$$

的球 x_i 的数量最多为 $\lceil n/m \rceil = O(1)$ 。这意味着任何给定桶中的球数也是 $O(1)$ 。由于解引用表以关于 n 的高概率成功，我们可以推导出以关于 n 的高概率，最满桶中的球数为 $O(1)$ 。这与最满桶中球数为 $\omega(1)$ 的事实矛盾，从而通过反证完成了证明。□

6 6 将微型指针应用于数据结构中的五个问题

在本节中，我们介绍微型指针在数据结构经典问题中的若干应用：

- **放松检索**：我们展示对经典检索问题的轻微修改消除了经典的每项 $\Omega(\log \log n)$ 浪费比特下界（第 6.2 节）。
- **简洁二叉搜索树**：我们给出了一种将任意动态二叉搜索树转换为简洁数据结构的方法（第 6.3 节）。
- **空间高效的稳定字典**：我们将任何固定容量键值字典转换为稳定键值字典（第 6.4 节）。
- **空间高效的字典**：我们将任何具有固定大小值的字典转换为能够空间高效地存储可变大小值的字典（第 6.5 节）。
- **最优内部内存暂存区**：我们构造了一个常数时间暂存区，空间高效地存储驻留在大型外部内存数据结构中的元素的位置（第 6.6 节）。

6.1 6.1 使用微型指针的通用技术

在深入具体应用之前，我们简要讨论几个预备定义和技术，它们在多个应用中都有用。

键值字典。我们的几个应用将执行黑盒变换，以为键值字典添加新特性（即稳定性和可变大小值）。形式上，**键值字典**（常简称为**字典**）是任何存储键值对的数据结构（例如哈希表或树），其中每个键最多出现一次。通常，键值字典支持插入、删除和查询，其中查询特别地返回与某个键关联的值。根据数据结构的不同，还可能支持额外的操作，例如后继查询（返回某个键的后继）。

我们说一个键值字典使用**值数组**（value array），如果它指定了某块连续内存（可随时间扩展或收缩），其目的是存储对应于键的值。如果值为 k 比特长，则值数组可视为 k 比特对象的数组。

在我们的应用中，我们将限制自己于将值存储在值数组中的字典。为简单起见，我们将假设字典使用单个值数组，尽管我们所有的结果也可以轻松地应用于使用多个单独

分配的值数组的字典（只要每个单独值数组至少为 $\Omega(\log n)$ 比特）。我们假设单个值数组的原因是，据我们所知，所有已知的空间高效键值字典都可以轻松地以此格式实现，因此我们选择避免为结果引入不必要的复杂性。

如何存储微型指针的值数组。我们几个应用中的一个主题是修改值数组，使其不直接存储值，而是存储某个大小 k 的微型指针。然而，回忆大小为 $k = o(\log \log \log n)$ 比特的微型指针不是固定大小的，意味着某些微型指针可能需要超过 k 比特。尽管如此，如果我们愿意使用常数因子更大的值数组，则有一个简单的技巧，我们称为**分块指针存储** (chunked pointer storage)，它使我们能够以与固定长度微型指针相同的方式与这些可变长度微型指针交互。

将值数组分解为 $O(\log n/k)$ 个微型指针的连续块。根据命题 2，每个块中微型指针使用的总比特数关于 n 以高概率为 $O(\log n)$ 。因此，每个块可以存储在 $O(\log n)$ 比特中，这意味着整个值数组可以存储在 $O(nk)$ 比特中。

然而，仍然存在如何高效地访问和修改给定块中第 j 个微型指针的问题。对每个块，我们可以存储一个额外的 $O(\log n)$ 比特位图，其中设为 1 的比特指示块中微型指针开始和结束的位置。为高效地找到第 j 个微型指针，只需在位图中找到第 j 个和第 $j+1$ 个 1。（然后可以使用标准位操作在位图和块上以常数时间提取、修改和重新插入微型指针。）在 $O(\log n)$ 比特位图中找到第 j 个 1 的问题可以用四俄罗斯方法 [9] 轻松解决：简单地存储一个大小为 $\sqrt{n}$ 的辅助查找表，允许在 $(\log n)/2$ 比特位图中通过单次查找回答此类查询，然后执行 $O(1)$ 次查找以在 $O(\log n)$ 比特位图中执行此类查询。

如何动态调整使用微型指针的数据结构的大小。我们的几个应用还会遇到在大小随时间动态变化的数据结构中使用微型指针的问题。当然，这意味着我们也必须动态调整解引用表的大小。我们的应用将采用以下方法，我们称为**分区聚合调整大小** (zone-aggregated resizing)。

考虑一个值数组，存储指向解引用表中 k 比特项的微型指针（假设 k 比特适合 $O(1)$ 个机器字）。假设我们希望将解引用表维持在 $1 - \Theta(1/k)$ 的负载因子，这样每项浪费的比特数为 $O(1)$ ；注意这意味着值数组中的微型指针平均为 $\Theta(\log k)$ 比特。然而，进一步假设值数组的大小随时间动态变化（意味着元素必须从解引用表中添加和移除）。在此讨论中，我们将假设值数组本身被动态调整大小，以始终处于至少 $\Theta(1)$ 的负载因子。

在项数随时间变化的情况下，我们如何更新解引用表以维持 $1 - \Theta(1/k)$ 的负载因子？我们不只使用单个解引用表，而是使用 k 个解引用表，并向每个微型指针添加 $\Theta(\log k)$ 比特以指示指向哪个解引用表（这不改变微型指针的渐近大小）。我们可以通过以下方式增长和收缩解引用表的容量（即槽位数）：(a) 重建最小的解引用表以将其大小加倍，或 (b) 重建最大的解引用表以将其大小减半。如果我们暂时假设重建解引用表花费的时间与表的大小成正比，则重建可以平摊化，使得每次操作（即对解引用表的每次修改）花费 $O(1)$ 时间，同时维持所需的 $1 - \Theta(1/k)$ 负载因子。

重建解引用表的问题是，所有指向该解引用表的微型指针都变为无效。新解引用表

的实际构造可以轻松地在线性时间内执行，但我们如何更新值数组中的微型指针？如果值数组大小为 n ，则被重建的解引用表仅包含 $\Theta(n/k)$ 个项。我们希望以 $\Theta(n/k)$ 时间（而非 $\Theta(n)$ 时间）标识指向这些项的微型指针在值数组中的位置。

此问题的解决方案非常简单：将值数组分解为连续的区域（zone），每个区域由 k 个值组成。在每个区域内，维护 k 个链表，其中第 i 个链表包含指向第 i 个解引用表的微型指针。重要的是，因为这些链表位于大小为 k 的区域内，每个链表内的指针仅需要 $\Theta(\log k)$ 比特；因此，链表不会渐近地增加值数组的大小。另一方面，为找到给定解引用表的所有微型指针，只需查看 $\Theta(n/k)$ 个区域中的每个区域的一个链表，使得所有 $\Theta(n/k)$ 个微型指针能够在 $\Theta(n/k)$ 时间内被识别。

出于我们稍后将看到的原因，我们的一个应用还将要求我们使用大小为 $\text{poly}(k)$ 而非仅为 k 的更大区域。现在，我们简单指出，使用大小为 $\text{poly}(k)$ 的更大区域仍然允许将每个微型指针的链表开销界定为 $\Theta(\log k)$ 比特，并且标识指向大小为 j 的解引用表的微型指针所需的时间仅为

$$O(j + n/\text{poly}(k)),$$

因为必须检查的链表数量仅为 $O(n/\text{poly}(k))$ 。

6.2 6.2 克服数据检索的 $\Omega(\log \log n)$ 比特下界

我们的第一个应用重新审视经典的**检索问题** [5, 26–28]，其中一个数据结构必须为某个集合 S 中每个 k 比特键存储一个 v 比特值，并且必须回答检索与给定键关联的值的查询。这里，我们处理该问题的动态版本，其中数据结构必须支持函数 $\text{Insert}(x, y)$ （向 S 中插入新 $x \in [2^k]$ 并关联值 $y \in [2^v]$ ）、 $\text{Delete}(x)$ （从 S 中移除某个 $x \in S$ ）和 $\text{Query}(x)$ （对某个 $x \in S$ 返回 x 对应的值 y ），允许集合 S 增长到某个最大大小 n 。注意，在检索问题中，用户有责任确保每次 Insert 调用在键 $x \notin S$ 上进行，每次 Query 调用在键 $x \in S$ 上进行，以及每次 Delete 调用在键 $x \in S$ 上进行。

已知如果 $k = \Omega(1 + \Omega(1)) \log n$ 比特，则动态检索问题的任何解必须使用至少 $nv + \Omega(n \log \log n)$ 比特空间 [5]，无论时间复杂度如何，即使 $v = 1$ 。进一步已知，如果 $k = \Theta(\log n)$ 且 $v = O(\log n)$ ，则 $nv + \Omega(n \log \log n)$ 空间界可以通过随机化常数时间数据结构实现 [26]。

我们现在将展示，通过稍微放松检索问题，我们可以使用微型指针获得显著更好的空间界。在**放松检索问题**（relaxed retrieval problem）中，插入/删除/查询操作被修改为如下工作。操作 $\text{Insert}(x, y)$ 现在返回一个**微型检索器**（tiny retriever） r ，用户必须记住它。将来，如果用户希望查询 x （且尚未删除 x ），他们调用 $\text{Query}(x, r)$ 以获得值 y 。最后，如果用户希望将 x 从集合 S 中移除，则用户调用 $\text{Delete}(x, r)$ 。

微型检索器的角色类似于微型指针——它充当辅助数据结构的提示。然而，与微型指针不同，对 (x, r) 不必完全编码 y 的位置；相反，查询操作 $\text{Query}(x, r)$ 可以使用辅

助元数据（除了 x 和 r 之外）来确定值 y 。我们现在将看到，这一区分非常重要，使得我们能够同时超越检索问题的下界 [5] 和微型指针问题的下界（定理 3）。同时（几乎矛盾地），正是我们对可变大小微型指针的构造使我们能够绕过这两个下界。

定理 6. 考虑具有 k 比特键、 v 比特值和最大容量 n 个键/值对的放松检索问题。令 $r \in [\log n]$ 为参数。存在放松检索问题的一个解，使用期望大小 $O(1)$ 的微型检索器，并以关于 n 的高概率：每次查询花费常数时间，每次插入/删除花费 $O(r)$ 时间，使用总空间 $nv + O(n \log^{(r)} n)$ 比特。

此外，如果 $\log^{(r)} n = \Theta(1)$ 且 $v \leq \frac{\log n}{\log^{(r)} n}$ ，则空间消耗变为 $nv + O(n)$ 比特。

上述定理提供了一个有趣的权衡曲线：常数时间插入/删除可以实现例如 $nv + O(n \log \log \log \log \log n)$ 比特的空间消耗，而 $O(\log^* n)$ 时间的插入/删除可以实现 $nv + O(n)$ 比特的空间消耗。此外，如果 v 略微亚对数，则即使是常数时间插入/删除也可以实现 $nv + O(n)$ 比特。

我们指出，定理 6 中的微型检索器实际上是如定理 2 中构造的可变大小微型指针。因此，它们满足定理 2 给出的双重指数尾不等式，以及命题 2 给出的集中不等式。

证明。 我们将利用定理 2 构造一个具有 $2n$ 个槽位的解引用表 T 。然而，我们对定理 2 的应用不寻常之处在于，我们将不在存储区中存储任何东西（事实上，我们甚至不需要为其分配空间）。相反，我们将利用 $\text{Dereference}(x, p)$ 是已被唯一分配给 x 的 $(1 + \log n)$ 比特数字这一事实。

为实现操作 $\text{Insert}(x, y)$ ，我们调用 $\text{Allocate}(x)$ 获得期望大小 $O(1)$ 的微型指针 p （注意 p 也将是我们的微型检索器）。定义 $s_x = \text{Dereference}(x, p)$ 为在 $[2n]$ 中分配给 x 的槽位号。我们将利用的主要性质是，对所有其他 $x' \in S$ 有 $s_x \neq s_{x'}$ 。为完成 Insert 操作，我们将键/值对 (s_x, y) 插入一个简洁哈希表 H （其规格我们稍后描述）。查询和删除则如下实现： $\text{Query}(x, p)$ 返回 $H[\text{Dereference}(x, p)]$ ； $\text{Delete}(x, p)$ 从 H 中删除键 $\text{Dereference}(x, p)$ 并对解引用表 T 调用 $\text{Free}(x, p)$ 。

数据结构的正确性源于以下事实：对每个 $x \in S$ 具有微型检索器 p ， $\text{Dereference}(x, p)$ 是唯一的。解引用表仅使用 $O(n)$ 比特空间，并以高概率支持常数时间操作。因此，为证明定理，尚需分析哈希表 H 。

我们使用已知最节省空间的哈希表构造来构造 H [13]。如果 H 存储来自全空间 U 的最多 n 个键且值为 v 比特，则它以高概率支持以下保证：查询为常数时间，插入/删除花费 $O(r)$ 时间，总空间消耗为

$$\log \binom{|U|}{n} + nv + O(n \log^{(r)} n)$$

比特。此外，如果 $\log^{(r)} n = \Theta(1)$ 且 $v \leq \frac{\log n}{\log^{(r)} n}$ ，则空间变为 $\log \binom{|U|}{n} + nv + O(n)$ 比特。

我们对微型指针的使用确保 H 中的键来自非常小的全空间 $U = [2n]$ 。因此

$$\log \binom{|U|}{n} = \log \binom{2n}{n} = O(n)$$

由 Stirling 近似。这完成了定理的证明。□

关于调整大小的说明。在第 6.3 小节中，我们将看到微型检索器在构造简洁二叉搜索树问题中的应用。在此应用中，我们将希望有两个放松检索数据结构，其大小之和最多为 n 。这里，我们可以利用上面使用的哈希表 H 实际上提供了动态调整大小保证这一事实：如果在任何给定时刻，哈希表的大小为 m ，则它最多使用

$$\log \binom{2n}{m} + mv + O(m \log^{(r)} n)$$

比特空间，以关于 n 的高概率。完整的检索数据结构（由哈希表 H 和解引用表 T 组成）因此最多使用

$$\log \binom{2n}{m} + mv + O(n + m \log^{(r)} n)$$

比特。由 Stirling 不等式，这最多为

$$m \log n - m \log m + mv + O(n + m \log^{(r)} n).$$

因此，如果我们有二个放松检索数据结构，一个大小为 $m_1 \leq n$ ，另一个大小为 $m_2 \leq n$ ，且 $m = m_1 + m_2 = \Theta(n)$ ，则它们的总空间消耗最多为

$$\begin{aligned} & (m_1 + m_2) \log n - m_1 \log m_1 - m_2 \log m_2 + (m_1 + m_2)v + O((m_1 + m_2) \log^{(r)} n) \\ &= m \log n - m_1 \log m_1 - m_2 \log m_2 + mv + O(m \log^{(r)} n). \end{aligned}$$

由 Jensen 不等式， $m_1 \log m_1 + m_2 \log m_2 \geq (m_1 + m_2) \log \frac{m_1 + m_2}{2} = m \log \frac{m}{2} = m \log n - O(n)$ 。因此总空间最多为

$$\begin{aligned} & m \log n - (m \log n - O(n)) + mv + O(m \log^{(r)} n) \\ &= mv + O(m \log^{(r)} n) \\ &= mv + O(m \log^{(r)} m). \end{aligned}$$

当然，这与我们为大小为 m 的单个放松检索器数据结构得到的界相同。

这之所以重要，是因为它提供了一种执行动态调整大小的简单方法：每次数据结构的大小 m 变化一个常数因子时，我们将当前放松检索数据结构 D_1 中的所有元素移动到新的放松检索数据结构 D_2 （参数化为基于 m 的新值具有容量 $n' = \Theta(m)$ ）。当我们将元素从 D_1 移动到 D_2 时， D_1 和 D_2 的总空间消耗将继续为 $mv + O(m \log^{(r)} m)$ 比特。

注意, 要将一个元素从 D_1 移动到 D_2 , 我们需要为该元素生成一个新的微型检索器 (因为我们正在从 D_1 中删除该元素并将其插入 D_2)。在我们的二叉搜索树应用中, 这可以通过简单地遍历所有元素并逐个重新定位它们轻松完成。此外, 由于构造 D_2 的工作可以分散到 $\Theta(n)$ 次操作中, 它可以以每次插入/删除 $O(r)$ 的代价实现。

6.3 6.3 简洁二叉搜索树

我们的第二个应用是一种将动态二叉搜索树转换为简洁数据结构的黑盒方法。如果简洁搜索树中有 n 个元素, 每个 k 比特长, 则简洁搜索树的大小最多为 $nk + O(n + n \log^{(r)} n)$ 比特, 其中 $r > 0$ 为任意参数。树中的路径遍历仅引入常数因子开销, 树的修改仅引入 $O(r)$ 因子开销。

我们方法的一个优势是它可以应用于基于旋转的搜索树。这包括例如红黑树 [36]、splay 树 [54] 等。如果动态最优性猜想 [54] 为真 (意味着 splay 树是动态最优的), 那么当 $r = O(1)$ 时, 我们的简洁 splay 树也是动态最优的。

定理 7. 考虑任何存储 a 比特键和 b 比特值的二叉搜索树, 其中每个节点关联一个不同的键, 且每个节点有指向其孩子的指针。对于任意 $r > 0$, 该树可以实现为以高概率 (关于树大小 n) 提供以下保证: 树占用 $na + nb + O(n + n \log^{(r)} n)$ 比特空间, 从父母到孩子的遍历需要 $O(1)$ 时间, 对树的修改 (即添加或删除边) 需要 $O(r)$ 时间。

我们指出, 信息论上, 树应消耗 $n(a + b)$ 比特空间。且由于键是唯一的, $na = \Omega(n \log n)$ 。因此, 对任意 $r > 1$, 上述搜索树是简洁的。

证明. 我们将利用我们对放松检索问题的解 (定理 6)。然而, 我们将存储在放松检索数据结构中的键/值对 (x, y) 会有些不寻常, 因为 y 将采取以下形式: y 包含 x 的 b 比特值, 以及两个微型检索器 r_1 和 r_2 。由于 r_1 和 r_2 本身是期望大小 $O(1)$ 的可变长度微型指针, 这意味着 y 也是可变长度的。另一方面, 放松检索数据结构是为固定长度值设计的。幸运的是, 我们可以用以下方法存储微型检索器 r_1 和 r_2 。回忆, 在我们对放松检索问题的构造中, 我们创建了一个具有 $2n$ 个槽位的解引用表, 但我们实际上不在解引用表的存储区中存储任何东西。我们现在对此进行更改, 使得存储区是一个具有 $2n$ 个槽位的值数组, 为解引用表中的每个项存储微型检索器 r_1 和 r_2 (所以, 如果 p 是 x 的微型指针, 则 r_1, r_2 位于值数组中第 $\text{Dereference}(x, p)$ 个位置)。使用分块指针存储技术, 我们可以确保值数组的总大小为 $O(n)$ 比特, 即使它存储的指针是可变长度的。

现在我们描述二叉搜索树的编码: 搜索树中的每个节点存储对应于该节点的键值对 (x, y) 以及两个微型检索器 r_1 和 r_2 。微型检索器 r_1 用于左孩子, 使用 $x \circ 0$ 作为其键 (因此 $\text{Query}(x \circ 0, r_1)$ 返回 x 的左孩子), 微型检索器 r_2 用于右孩子, 使用 $x \circ 1$ 作为其键 (因此 $\text{Query}(x \circ 1, r_2)$ 返回 x 的右孩子)。注意, 如果左孩子 (或右孩子) 不存在, 则我们简单地将 r_1 (或 r_2) 设为 null 。

让我们首先假设我们的二叉搜索树具有 n 个键/值的固定容量, 因此我们可以使用容量为 n 的放松检索数据结构。那么我们的放松检索数据结构使用 $na + nb + O(n +$

$n \log^{(r)} n$ 比特。从节点导航到其孩子花费 $O(1)$ 时间（因为它需要对放松检索数据结构的单次查询），添加/删除从节点 x 到孩子 z 的边 (x, z) 以高概率花费 $O(r)$ 时间，因为它仅需要对放松检索数据结构的单次插入/删除；重要的是，如果 z 是某子树的根，将 z 设为 x 的孩子的操作不要求除 z 外的任何节点在放松检索数据结构中被插入/删除。

最后，让我们修改数据结构，使其作为当前键/值对数量 n 的函数动态调整大小。为此，我们可以简单地使用第 6.2 节概述的调整大小方法。每次 n 变化一个常数因子时，我们重建放松检索数据结构，使其对新值 n 具有容量 $\Theta(n)$ 。（注意这不要求我们重建树；它只要求我们更新每个节点中使用的微型检索器。）对二叉搜索树中的每个放松检索器，我们可以存储一个额外比特，指示它使用两个放松检索数据结构中的哪一个——这保持了正确性。如第 6.2 节中观察到的，将项从旧放松检索数据结构移动到新数据结构不会违反我们所需的空间保证：我们的搜索树使用的总比特数始终保持为 $na + nb + O(n + n \log^{(r)} n)$ 。并且，通过将重建放松检索数据结构的工作分散到 $\Theta(n)$ 次操作中，我们保持了每次边插入/删除花费 $O(r)$ 时间的性质。因此定理得证。□

6.4 6.4 空间高效稳定字典

使用微型指针，我们给出一种将任何固定容量键值字典转换为**稳定字典**的黑盒方法，意味着值被存储的位置在值被插入后永不改变。如果原始字典存储 k 比特值，则新字典也存储 k 比特值，并且每个值使用的额外空间最多比原始数据结构多 $O(\log k)$ 比特。

定理 8。考虑一个将其值存储在某个大小为 m 的值数组中的固定容量键值字典数据结构 T 。令 v 表示每个值的比特大小。

可以构造一个新的数据结构 T' ，具有与 T 相同的操作和渐近性（以高概率），但额外具有 T' 是**稳定的**这一性质。此外， T' 消耗的总空间比 T 最多多 $O(m \log v)$ 比特（以关于 m 的高概率保证）。

证明。为构造 T' ，我们简单地将 T 的值数组替换为 m 个微型指针的数组，每个大小为 $\Theta(\log v)$ 比特。（如果 $\log v < \log \log \log n$ ，则可以使用分块存储技术处理不同微型指针具有不同大小的事实。）微型指针指向一个大小为 $(1 + 1/v)m$ 的解引用表，该表存储 m 个 v 比特值。（因此负载因子为 $1 - \Theta(1/v)$ 。）如果微型指针指向对应于键 x 的值 y ，则该微型指针使用 x 作为其键。这确保了稳定性，因为即使微型指针被存储的位置发生变化，微型指针也不必改变（且值 y 不必移动）。

微型指针数组消耗 $O(m \log v)$ 空间。而 T 中的值数组消耗 mv 比特， T' 中的解引用表消耗 $(1 + 1/v)mv$ 比特，仅比 T 多 $O(m)$ 比特。因此关于空间效率的论断得到证明。由于微型指针仅为每次访问/修改值增加常数时间， T 和 T' 的渐近性（以关于 m 的高概率）相同。□

6.5 6.5 具有可变大小值的空间高效字典

我们的第四个应用是一种将任何键值字典（设计用于存储固定大小值）转换为可以存储不同键的不同大小值的字典的黑盒方法。由此产生的数据结构提供以下关于空间效率的引人注目的保证。令 $\log^{(r)} n = \log \log \cdots \log n$ 表示 n 的第 r 次迭代对数。令 r 为我们选择的一个正常数，令 m 为原始字典使用的值数组中的条目数（在某个给定时刻）。允许值为任意长度的新字典将 T 的值数组替换为消耗最多

$$O(m \log^{(r)} m) + \sum_{i=1}^m (v_i + O(\log v_i))$$

比特的数据结构，其中 $v_1, v_2, \dots, v_m$ 表示正在存储的值的比特长度。

定理 9。 考虑一个将其值存储在值数组中，且设计用于存储固定长度键的键值字典数据结构 T 。令 r 为我们选择的一个正常数。

可以构造一个新的数据结构 T' ，具有与 T 相同的操作和渐近性（以高概率），但额外具有 T' 可以存储任意长度的值（最高到 $O(1)$ 个机器字）的性质。

在任何给定时刻，如果 T 原本使用大小为 m 的值数组，且机器字大小 w 满足 $w \geq m^{o(1)}$ ，则 T' 为实现值数组消耗的总空间保证（以关于 m 的高概率）最多为

$$O(m \log^{(r)} m) + \sum_{i=1}^m (v_i + O(\log v_i))$$

比特，其中 $v_1, v_2, \dots$ 是值的大小。

我们指出，对值大小限制为 $O(1)$ 个机器字仅仅是为了使每个值可以在常数时间内读写，这样便于讨论 T 和 T' 的渐近性如何比较。相同的技术对更大的值也可以不加修改地工作，只要愿意花费必要的时间来读写超常数大小的值。

定理 9 的证明。 T' 中的值通过最多 r 层间接引用存储。如果一个值为 k 比特，则它被一个大小为 $O(\log k)$ 比特的微型指针 p_1 指向。微型指针 p_1 依次被一个大小为 $O(\log \log k)$ 比特的微型指针 p_2 指向，依此类推，指针大小为 $O(\log \log \log k), O(\log \log \log \log k), \dots, O(1)$ 即，每个值存储在长度为 $O(1)$ 的链表的末端，其中链表的基指针为 $O(\log^{(r)} n)$ 比特，每个后续指针是指数级大于前一个指针的。

对数据结构中某个大小为 j 的每个微型指针，我们还必须存储 $O(j)$ 个额外信息比特，指示 (a) 该微型指针是指向另一个微型指针还是指向一个最终值，以及 (b) 被指向的微型指针/值的大小是什么。在证明的其余部分，我们将把这些 $O(j)$ 额外比特计入微型指针的大小。

由于存在多种不同大小的值和微型指针，我们必须为每种大小类别的微型指针使用不同的解引用表，并为每种大小类别的被存储值使用不同的解引用表。（注意，存储微型指针的解引用表可能需要使用分块存储技术来处理可变大小的微型指针，所以不应使用同一个解引用表来存储微型指针和值。）

同时动态调整所有解引用表大小的问题略显微妙。考虑一个解引用表 A (A 也可以是值数组)，它为某个 j 存储 j 比特的微型指针。有 $K = 2^{\Theta(j)}$ 个不同的解引用表 $B_1, B_2, \dots, B_K$ ，这些微型指针可以指向它们（取决于被指向对象的大小，以及该对象是微型指针还是值）。每个 B_i 必须单独进行动态大小调整。我们将维护所谓的**动态大小调整不变量** (dynamic-sizing invariant)，它保证每个 B_i 要么 (a) 处于负载因子 $1 - O(1/j)$ (其中 j 是存储在 B_i 中对象的大小)，要么 (b) 大小 (以比特计) 最多为 A 的 $o(1/(Kj))$ 分之一。

为实现动态大小调整不变量，我们使用分区聚合调整大小动态调整每个 B_i 的大小 (回忆第 6.1 节，这意味着 B_i 被分解为多个组件，每个组件偶尔被重建以使其大小加倍或减半)。为允许每个 B_i 的组件被高效重建，我们将 A 分解为大小为 $\text{poly}(K)$ 的区域，根据第 6.1 节的 (3)，这意味着某个 B_i 的由 s 个条目组成的给定组件可以在时间

$$|A|/\text{poly}(K) + s$$

内重建，其中 $|A|$ 是 A 中的条目数。我们对 B_i 执行动态大小调整的方式取决于它是否非常小 (其组件每个包含少于 $|A|/\text{poly}(K)$ 个元素)：

- **如果组件包含 $s = \Omega(|A|/\text{poly}(K))$ 个元素**，则我们执行分区聚合调整大小 (完全如第 6.1 节所述) 以保持 B_i 处于负载因子 $1 - O(1/j)$ ，其中 j 是存储在 B_i 中对象的大小。在这种情况下，重建大小为 s 的组件所需的时间为 $\Theta(s)$ ，因此 B_i 的动态大小调整可以平摊化为花费 $O(1)$ 时间 (每次在 B_i 上的操作)。注意，这里 B_i 处于动态大小调整不变量的情况 (a)。
- **如果组件每个包含少于 $|A|/\text{poly}(K)$ 个元素**，则我们执行分区聚合调整大小以保持 B_i 的每个组件处于容量 $\Theta(|A|/\text{poly}(K))$ (即使 $|A|$ 随时间变化，并且无论每个组件的元素数是否可能显著小于 $|A|/\text{poly}(K)$)。注意，这里 B_i 处于动态大小调整不变量的情况 (b)。

当 B_i 处于此状态时，我们不能将花费在重建 B_i 上的工作分摊到对 B_i 执行的操作上。相反，我们以下列方式分散花费在重建 B_i 组件上的工作：对在 A 上花费的每 $\Theta(K)$ 工作量，我们也在调整 B_i 大小上花费 $O(1)$ 时间。由于 B_i 比 A 小超过 K 的因子，这足以保持 B_i 处于每个组件具有容量 $\Theta(|A|/\text{poly}(K))$ 的状态。

从 A 的角度看，每次我们在插入/删除/重建 A 上花费常数时间时，我们也可能在某个 B_i 上花费常数时间执行重建工作 (这反过来可能递归地导致我们在 B_i 指向的解引用表之一上花费常数时间进行重建，等等)。重要的是，由于微型指针链最多 $r = O(1)$ 长，花费在重建上的时间仅为每次操作的运行时间引入常数因子开销。

上述调整大小方法保证了动态大小调整不变量，同时每次操作仅引入常数因子时间开销。接下来我们使用不变量来界定 T' 的空间消耗。处于情况 (a) 的解引用表 B_i 实现

得足够空间高效，使得它们中的空槽位相比于实际存储在其中的对象占有可忽略的空间（即空槽位每个对象增加 $O(1)$ 比特），而处于情况 (b) 的解引用表 B_i 则足够小，使得相比于父解引用表 A 的大小它们占有可忽略的空间（即它们在 A 中每个槽位累计增加 $o(1)$ 比特）。由此得出，解引用表消耗的总空间最多为存储在解引用表中对象的大小之和，加上每个对象 $O(1)$ 比特；这反过来意味着 T' 用于存储值/微型指针的空间由 (4) 给出。

接下来，我们界定 T' 与 T 相比的时间开销。我们已经证明，在解引用表上执行动态大小调整的时间开销为每次操作 $O(1)$ 。由于值通过最多 $r = O(1)$ 层间接引用存储，访问/修改一个值所需的时间也是 $O(1)$ 。因此 T' 具有与 T 相同的时间渐近性。

最后，我们论证 T' 使用的解引用表以高概率在其分配上成功。¹⁰ 对此我们可以采用几种方法；最简单的是对我们执行解引用表大小调整的方式再增加一项修改：**每当一个解引用表达达到大小 $\Theta(\sqrt{m})$ 时，我们不再将其调整得更小。**¹¹ 这意味着某些解引用表可能非常稀疏，包含 $\Theta(\sqrt{m})$ 个槽位但包含远更少的元素。由于只有 $O(w) = m^{o(1)}$ 个不同的解引用表（回忆 w 是机器字大小），大小为 $\Theta(\sqrt{m})$ 的解引用表的净空间消耗为 $o(m)$ 比特。每个解引用表具有至少 $\Omega(\sqrt{m})$ 的大小的这一事实意味着所有解引用表提供高概率保证，正如所需。□

¹⁰ 有多种不同的处理分配失败的方法，包括例如对数据结构执行批量重建。

¹¹ 然而，由于 m 可能随时间动态变化，我们确实需要每次操作花费常数时间调整大小为 $\Theta(\sqrt{m})$ 的解引用表，使得它们随着 m 变化而保持大小为 $\Theta(\sqrt{m})$ 。

6.6 6.6 最优内部内存暂存区

我们的最后一个微型指针应用重新审视外部内存数据结构中最古老的问题之一：维护一个小型内部内存暂存区 (stash)，允许定位元素在大型外部内存数据结构中的位置。

该问题可以形式化如下。我们必须存储最多 n 个键值对的动态变化集合 S ，每个键值对可以存储在一个机器字中，每个键是唯一的。我们被给予一个由 $(1 + \epsilon)n$ 个机器字组成的外部内存，键值对 S 将存储在其中。除了在外部内存中存储键值对外，我们还必须维护一个小型内部内存数据结构 X （我们称之为**暂存区**），支持以下操作：

- $\text{Query}(k)$ ：仅使用暂存区数据结构中的信息，返回键 k 及其对应值 v 存储在外部内存中的位置。
- $\text{Insert}(k, v)$ ：插入键值对 (k, v) ，将该对放置在外部内存中的某处，并更新暂存区。
- $\text{Delete}(k, v)$ ：从外部内存数组中移除键/值对 (k, v) ，并更新暂存区。

暂存区的重要特点是查询可以通过对**外部内存的单次访问**完成。另一方面，为了使暂存区有用，还必须实现几个其他目标：

- **紧凑性**：暂存区 X 需要尽可能小，这样才能放入大小有限的内部内存中。
- **高效的插入和删除**：虽然暂存区优先考虑查询，但插入和删除理想情况下也应仅需要对内部内存的 $O(1)$ 次访问/修改。
- **RAM 效率**：最后，为了使计算开销不成为瓶颈，暂存区上的操作应在 RAM 模型中尽可能高效，理想情况下花费 $O(1)$ 时间。

在现实世界系统中使用的暂存区的一个具体例子是页表 [1, 2, 10]，它是一个操作系统级字典，将虚拟页地址映射到其对应物理页在内存中的位置。页表在每次地址转换时都被访问，因此它是性能关键的，从而被高度优化。此外，页表空间高效是重要的，以便它可以有效地缓存在处理器缓存层次结构中。注意，虽然页表可以选择物理页在内存中的驻留位置，但它们不能移动已经放置的物理页；因此任何用作页表的暂存区也必须是稳定的。出于这个原因，过去的工作 [35, 41, 42] 通常将稳定性作为暂存区的额外标准。

关于设计空间高效和时间高效暂存区的工作可追溯到 20 世纪 80 年代末 [35, 41, 42]。最著名的理论结果归功于 Gonnet 和 Larson [35]，他们给出了一个仅使用 $O(n \log \epsilon^{-1})$ 比特的稳定暂存区。一个引人注目的推论是，当 $\epsilon = \Theta(1)$ 时，可以仅使用 $O(n)$ 比特构造一个暂存区。

然而，Gonnet 和 Larson 的结果有几个已被证明难以修复的重大缺点 [35]。首先，由于其依赖于稳定均匀探测 [40] 作为确定键/值应驻留位置的机制，暂存区仅在插入/删除随机执行的情况下提供可证明保证。其次，该数据结构在 RAM 模型中不是常数时间的，而是花费期望时间 $\Theta(\epsilon^{-1})$ 。

使用微型指针，我们证明了用于构造过滤器的现代技术可以轻松地适配，以构造一个大小的稳定暂存区 $O(n \log \epsilon^{-1})$ 比特，在 RAM 模型中支持常数时间操作（以高概率），并支持任意的插入/删除/查询序列。

定理 10. 可以构造一个稳定暂存区，在 RAM 模型中支持常数时间操作，将最多 m 个键/值存储在大小为 $(1 + \epsilon)m$ 的外部内存数组中，并仅使用 $O(m \log \epsilon^{-1})$ 比特的内部内存空间。暂存区的所有保证以关于 m 的高概率成立。

证明。 我们设计的起点是 Bender 等人 [12] 的**自适应过滤器** (adaptive filter)。与暂存区类似，他们的过滤器是一种空间高效的内部内存数据结构，总结外部内存键值字典的状态。与暂存区不同，他们的过滤器不指示每个键/值存储在外部内存中的位置。相反，过滤器以以下保证回答包含查询：每个正查询保证返回 true，每个负查询保证以至少 $1 - \epsilon$ 的概率（对某个参数 ϵ ）返回 false。其内部内存数据结构的大小仅为 $(1 + o(1))m \log \epsilon^{-1} = O(m \log \epsilon^{-1})$ 比特，其中 m 是过滤器的容量。¹²

¹² 事实上，他们的数据结构也是动态可调整大小的，但对于我们的应用这并非必要。

[12] 的自适应过滤器背后的基本思想是为每个键 x 存储一个**指纹** (fingerprint)，其中每个指纹取为哈希 $h(x)$ 的某个前缀。不同的键具有不同长度的指纹，过滤器维护的

不变量是没有指纹是任何其他指纹的前缀。为了在维护此不变量的同时保持指纹尽可能小，过滤器有时会在给定插入/删除过程中改变 $O(1)$ 个不同指纹的长度；为了改变指纹的长度，必须首先从外部内存获取对应于该指纹的键，这样才能重新计算该键的哈希 $h(x)$ 。¹³

¹³ 原始数据结构有时也在负查询期间更新指纹的长度，但此类更新对于我们的数据结构的目的不是必需的。

过滤器中的指纹存储如下。每个指纹的前 $\lg n$ 比特称为**商** (quotient)，这些比特用于将键分配到 n 个桶之一；重要的是，桶选择编码桶中每个键的商这一事实意味着数据结构不必显式地存储指纹的商。每个指纹接下来的 $\log \epsilon^{-1}$ 比特称为**基线比特** (baseline bits)，这些比特包含在数据结构中的每个指纹中。最后，指纹中的任何后续比特称为**自适应比特** (adaptivity bits)，这些比特被添加/移除以维护无前缀不变量。[12] 分析的中心部分是证明总共有 $O(m)$ 个自适应比特，并且这些比特可以被高效存储。

我们现在描述如何将过滤器修改为暂存区。除了为每个键存储指纹外，我们现在还存储一个期望大小为 $\Theta(\log \epsilon^{-1})$ 的微型指针。这些微型指针很容易存储，因为过滤器已经为每个键的 $\log \epsilon^{-1}$ 基线比特预留了空间。当然，不同的微型指针可能具有不同的长度，但此问题可以轻松地通过使用第 6.1 节描述的分块存储技术来解决（或通过适配 [12] 中已经用于处理可变长度指纹的技术）。

一个较小的困难是，过滤器假设可以访问外部内存字典（而不只是解引用表），这样才能查找键以修改其指纹。然而，在我们的暂存区情况下，这些查找可以轻松地使用已存储的微型指针来执行，因此不需要外部内存中的完整字典。

微型指针的大小为 $\Theta(\log \epsilon^{-1})$ 这一事实意味着外部内存可以实现为负载因子 $1 - \epsilon$ 的解引用表。原始自适应过滤器支持常数时间操作（以关于 m 的高概率）这一事实转化为暂存区也支持常数时间操作。原始自适应过滤器在内部内存中使用 $O(m \log \epsilon^{-1})$ 比特空间这一事实也转化为对暂存区的相同保证。因此定理得证。□

7 7 动态球与桶

在本节中，我们将微型指针构造重新解释为球与桶方案，以改进经典的动态负载均衡问题的最新技术水平。

在**动态负载均衡问题**中，有一个包含 n 个桶的系统和一个大的球的全空间 U 。球被健忘对手随时间插入和删除（有时重新插入），使得系统中的球总数不超过 $m = hn$ (h 为某参数)。每当一个球 x 被插入时，它必须被放置在 $\text{Bin}_1(x), \dots, \text{Bin}_d(x)$ 中的 d 个桶之一，其中 $\text{Bin}_i(\cdot)$ 是从球到桶的某个哈希函数。一旦球被放入桶中，在它被删除之前不能移动。动态负载均衡问题的目标是将球分配到桶中以实现尽可能小的最大负载（即最小化最满桶中的球数）。我们将球可以被插入和删除但不能重新插入的特殊情况称为**半动态负载均衡问题**。

该问题有两个经典解决方案。第一个是**单一球与桶分配** (Single): 我们设 $d = 1$, 仅将每个 x 放入 $h_1(x)$ 。第二个是 **Left[d] 球与桶分配**: 将桶分成 d 组, 使得每个 h_i 到第 i 组是均匀的; 当插入 x 时, 选择具有最小负载的桶 $h_i(x)$, 通过最小化 i 来打破平局。

Single 的行为是**历史独立的**, 在任何时候最大负载仅取决于哪些球存在, 而不取决于它们到达的历史。最大负载则由标准 Chernoff 界完全刻画。

另一方面, Left[d] 是**高度历史依赖的**。第一次插入球 x 时, 哈希 $\text{Bin}_1(x), \dots, \text{Bin}_d(x)$ 独立于系统状态, 但如果球 x 被删除然后又被重新插入, 则 x 的过去插入可能对系统状态产生长期副作用, 意味着状态不一定独立于 $\text{Bin}_1(x), \dots, \text{Bin}_d(x)$ 。

在仅插入的设置中 (即球不删除), Left[d] 提供了著名的界 [55]

$$h + \frac{\log \log n}{d \log \phi_d} + O(1)$$

关于最大负载, 其中 ϕ_d 是广义黄金比例。在动态设置中, Left[d] 被证明分析起来要困难得多。Vöcking [55] 对 Left[d] 的原始分析可用于在半动态设置中实现

$$O(h) + \frac{\log \log n}{d \log d}$$

的界, 但正如 Woelfel 观察到的 [56], 相同的论证不直接适用于完全动态设置。¹⁴ 他展示了如何修改 Vöcking 的证明以在 $h = 1$ 的设置中实现

$$O(d) + \frac{\log \log n}{d \log d}$$

的界。一般地, 当 $h > 0$ 时, Woelfel 的论证产生

$$O(1 + hd) + \frac{\log \log n}{d \log d},$$

这仍是当前的技术水平。

¹⁴ 困难与见证树中叶子的分析有关, 在 $h = 1$ 的情况下很容易描述。为分析叶子球 x , 原始分析使用 Markov 不等式推导出 x 的 d 个桶每个最多有 $1/3$ 的概率包含 3 个或更多球, 分析得出结论所有 d 个桶包含 3 个或更多球的概率最多为 $1/3^d$ 。相同的分析不适用于完全动态设置, 因为它需要系统状态独立于 x 的哈希函数 $\text{Bin}_1(x), \dots, \text{Bin}_d(x)$, 但由于系统状态中微妙的历史依赖关系, 情况并非如此。

界 (8) 在 h 相对较小的情况下最有趣, 即 $h = o(\log n)$ 。这里, (8) 可以显著好于 Single 会实现的 $\Theta(\log n / \log \log n)$ 界。当然, 问题仍然是是否存在能达到更好界的球与桶方案。我们以肯定的方式回答这个问题, 给出一个使用 $d + 1$ 个哈希函数的桶选择规则, 实现最大负载

$$h + \frac{\log \log n}{d \log d} + O\left(\sqrt{h \log(hd)}\right).$$

我们指出, 即使 h 是常数, 此界也将对 d 的依赖从 $O(d)$ 改进为 $O(\sqrt{\log d})$ 。

我们的规则, 我们称之为 **Iceberg** $[d]$, 是 **Single** 和 **Left** $[d]$ 的混合。此规则与我们在第 3 节中用于构造固定大小微型指针的规则密切相关。

本节剩余部分如下进行。我们在第 7.1 小节中首先证明一个有用的技术引理。在第 7.2 小节中, 我们提出并分析 **Iceberg** $[d]$ 。最后, 在第 7.3 小节中, 我们将可变大小微型指针构造重新解释为关于容量为 1 的桶的球与桶方案的探测复杂度的结果; 特别地, 我们给出第一个动态球分配方案, 在系统中一次最多有 $(1 - \delta)n$ 个球存在的设置中提供 $\text{poly}(\delta^{-1})$ 的平均探测复杂度。

7.1 7.1 一个有用的引理

本节证明由当前作者的一个子集在近期关于空间高效哈希表的工作 [11] 中引入的技术引理的一个推广。新引理将原始引理扩展到更广泛的参数范围。我们还采用了与之前论文不同的组合方法, 产生了一个更简单的证明, 揭示了该引理与 Talagrand 不等式之间的有趣关系。

考虑一个有 n 个桶且始终最多有 $m = hn$ 个球的动态球与桶博弈, 球使用 **Single** 规则放置。每当一个球被扔进一个桶时, 如果该桶包含 $h + \Delta$ 个或更多球, 则该球被标记为 Δ -**暴露**的 (Δ -exposed), 且该标签持续到该球下次被删除。

引理 3。假设 $1 \leq \Delta \leq h$ 。在任何固定的时间点, Δ -暴露球的数量为 $\text{poly}(h) \cdot n \cdot e^{-\Omega(\Delta^2/(3h))}$, 概率为 $1 - \exp(-\Omega(me^{-\Delta^2/(3h)}))$ 。

我们对引理的证明将使用 Talagrand 不等式的一个变体 [44, 第 12 章]:

定理 11 (Talagrand 不等式)。令 $X_1, \dots, X_n$ 为来自任意域的 n 个独立随机变量。令 F 为 $X_1, \dots, X_n$ 的函数, 不恒为 0。假设对某 $c, r > 0$, F 是 c -Lipschitz 且 r -可认证的, 定义如下:

- F 是 c -Lipschitz 的, 如果改变任何单个 X_i 的结果最多改变 F 为 c 。
- F 是 r -可认证 (r -certifiable) 的, 如果对任意 s , 若 $F(X_1, \dots, X_n) \geq s$, 则存在最多 rs 个 X_i 的认证集合, 其结果作为 $F \geq s$ 的见证, 即不论不在认证集合中的其他 X_j 的结果如何都有 $F \geq s$ 。

则对任意 $0 \leq t \leq \mathbb{E}[F]$,

$$\Pr \left[|F - \mathbb{E}[F]| > t + 60c\sqrt{r\mathbb{E}[F]} \right] \leq 4 \exp \left(-\frac{t^2}{8c^2r\mathbb{E}[F]} \right).$$

引理的证明通过界定暴露球的期望数量, 然后使用 Talagrand 不等式获得集中界来进行。

在下文中, 我们将最终时存在的球称为 $a_1, \dots, a_k$, 将全空间中剩余的球称为 $a_{k+1}, \dots, a_{|U|}$ 。我们用 σ_i 表示 a_i 的桶选择。对 $i \in [k]$, 我们定义 t_i 为 a_i 被插入的最后时间, 定义 X_i

为指示 a_i 在博弈结束时是否为暴露球的随机变量，并定义 $X = \sum_{i=1}^k X_i$ 为暴露球的总数。

论断 1。 暴露球的期望数量满足 $\mathbb{E}[X] = O(me^{-\Delta^2/(3h)})$ 。

证明。 回忆 $X = \sum_i X_i$ ，其中 X_i 指示 a_i 是否暴露。由期望的线性性，只需对每个 $i \in [k]$ 证明 $\mathbb{E}[X_i] = O(e^{-\Delta^2/(3h)})$ 。

固定 $i \in [k]$ 。考虑球 a_i 被插入的最后时间 t_i 。球 a_i 暴露当且仅当桶 σ_i 中的球数至少为 $h + \Delta$ 。如果我们设 Y 为桶 σ_i 中的球数，并设 $\gamma = \Delta/h$ ，则我们可以使用 Chernoff 界界定 $Y \geq h + \Delta$ 的概率：

$$\Pr[Y \geq h + \Delta] = \Pr[Y \geq (1 + \gamma)h] \leq e^{-\gamma^2 h/3} = e^{-\Delta^2/(3h)}.$$

因此 $\Pr[X_i] = e^{-\Delta^2/(3h)}$ 。□

论断 2。 随机变量 X 作为 $\{\sigma_i\}_{i=1}^{|U|}$ 的函数是 $(h + \Delta + 1)$ -Lipschitz 且 $(h + \Delta + 1)$ -可认证的。

证明。 将单个 σ_i 的值改为 σ'_i 只会影响桶 σ_i （可能减少）和桶 σ'_i （可能增加）中的暴露球数。桶中未暴露球的数量确定性地最多为 $h + \Delta$ 。这意味着将球 a_i 从桶 σ_i 中移出可以使该桶中未暴露球的数量最多增加 $h + \Delta$ ，因此可以使暴露球的数量最多减少 $h + \Delta + 1$ （其中 $+1$ 计及 a_i 本身的移除）。类似地，将球 a_i 移入桶 σ'_i 可以使该桶中未暴露球的数量最多减少 $h + \Delta$ ，因此可以使暴露球的数量最多增加 $h + \Delta + 1$ 。这证明了 X 是 $(h + \Delta + 1)$ -Lipschitz 的。

为认证 $X \geq s$ ，令 $J \subseteq [k]$ 满足 $|J| = s$ 为一个值 j 的集合，使得 a_j 在博弈结束时暴露。对每个 $j \in J$ ，令 R_j 为 $h + \Delta$ 个球 i 的选择，使得球 a_i 在 a_j 被插入的最后时间 t_j 时存在且 $\sigma_i = \sigma_j$ 。随机变量集合 $\{\sigma_i \mid i \in R_j\} \cup \{\sigma_j\}$ 作为 a_j 暴露的认证。因此集合

$$\bigcup_{j \in J} (\{\sigma_i \mid i \in R_j\} \cup \{\sigma_j\})$$

作为 $X \geq s$ 的认证。此认证由 $s(h + \Delta + 1)$ 个随机变量组成，因此 X 是 $(h + \Delta + 1)$ -可认证的。□

引理 3 的证明。 设 $Q = m \exp(-\Delta^2/(3h))$ 。由论断 1，我们知道 $\mathbb{E}[X] \leq Q$ 。由论断 2，我们可以对 X 应用 Talagrand 不等式（定理 11），其中 $c = r = h + \Delta + 1 = O(h)$ 。应用 Talagrand 不等式取 $t = \Theta(c\sqrt{rQ})$ ，并使用 Q 作为 $\mathbb{E}[X]$ 的上界，我们可以推导出

$$X = O(c\sqrt{rQ})$$

以至少

$$1 - \exp(-\Omega(Q))$$

的概率成立。由此得出 $X \leq \text{poly}(h) \cdot O(ne^{-\Delta^2/(3h)})$, 概率为 $1 - \exp(-\Omega(me^{-\Delta^2/(3h)}))$ 。

□

7.2 7.2 Iceberg[d]

我们现在提出 Iceberg[d] 球入桶规则。令 n 为桶的数量, 令 hn 为任何给定时刻允许存在的最大球数, 令 $d > 1$ 为参数。将桶划分为 d 个等大小的集合 $S_1, \dots, S_d$ 。令 g 为将球均匀随机映射到桶的哈希函数, 令 $h_1, \dots, h_d$ 为使得每个 h_i 将球均匀随机映射到 S_i 中随机桶的哈希函数。

我们将有三种类型的球: 一级球、二级球和三级球。每个一级球 x 将驻留在桶 $g(x)$ 中, 每个二级球 x 将驻留在桶 $h_1(x), \dots, h_d(x)$ 之一中, 每个三级球 x 将驻留在桶 1 中 (但在任何给定时刻, 三级球的数量以高概率为零)。

设 $\Delta = c\sqrt{h \log(hd)}$, 其中 c 为某个足够大的正常数。我们还将跟踪一个变量 q , 计数在任何给定时刻存在的二级球数量。

插入球 x 的过程如下。如果桶 $g(x)$ 包含 $h + \Delta$ 个或更少的一级球, 则我们将 x 放入桶 $g(x)$, 并将 x 分类为一级球。否则, 我们检查是否 $q < n/d$ 。如果 $q < n/d$, 则我们检查桶 $h_1(x), \dots, h_d(x)$, 并将 x 作为二级球放入包含最少二级球的桶 $h_i(x)$ (通过最小化 i 打破平局)。最后, 如果 $q \geq n/d$, 则我们将 x 作为三级球放入桶 1。

定理 12. 假设 $1 \leq h \leq n^{o(1)}$ 且 $1 < d \leq n^{o(1)}$ 。假设球随时间根据 Iceberg[d] 规则被插入/删除/重新插入到 n 个桶中 (由健忘对手), 一次最多有 hn 个球存在。那么, 关于 n 以高概率, 在任何给定时刻, 最满桶中的球数为

$$h + \frac{\log \log n}{d \log d} + O\left(\sqrt{h \log(hd)}\right).$$

证明。 每个桶确定性地包含最多 $h + \Delta = h + O(\sqrt{h \log(hd)})$ 个一级球。因此, 只需将每个桶中二级球和三级球的数量界定为 $\frac{\log \log n}{d \log d} + O(1)$ 。

整个系统中二级球的数量 q 在任何给定时刻确定性地最多为 n/d 。换句话说, 二级球根据 Left[d] 规则放置, 有 $h'n$ 个球, 其中 $h' = 1/d$ 。因此我们可以应用 (8) 推导出, 以高概率, 每个桶中此类球的最大数量为

$$O(1 + h'd) + \frac{\log \log n}{d \log d} = O(1) + \frac{\log \log n}{d \log d}.$$

注意, 在此应用 (8) 时, 我们在某种程度上不寻常的参数范围内使用 Woelfel [56] 对 Left[d] 的分析; 即, 该分析主要旨在 $h \geq 1$ 的范围内使用 (且 Woelfel 的结果仅对 $h = \Theta(1)$ 显式陈述), 但我们利用了该分析对 $h = o(1)$ 也不加修改地成立这一事实。

我们通过证明以高概率三级桶的数量为零来完成证明。由引理 3, 二级球的数量 q 满足 $q < n/h$ (在任何给定时刻), 概率至少为 $1 - \exp(-n/\text{poly}(hd))$, 由假设 $h, d \leq n^{o(1)}$ 这至少为 $1 - 1/\text{poly}(n)$ 。由此得出, 每次单个球插入以最多 $1/\text{poly}(n)$ 的概率成为三级

球。对系统中所有球取联合界，其中任何一个球成为三级球的概率为 $1/\text{poly}(n)$ ，正如所需。□

7.3 7.3 以低平均探测复杂度将球分配到容量为 1 的桶

本节的最后一个结果考虑一个动态球与桶博弈，其中有 n 个容量为 1 的桶，且一次最多有 $(1-\delta)n$ 个球存在。每个球 x 有一个预先确定的（无限）桶序列 $h_1(x), h_2(x), \dots$ ，它可以驻留在这些桶中，我们希望最小化每个球 x 的**探测复杂度**（probe complexity），它定义为满足球 x 在桶 $h_i(x)$ 中的最小 i 。由于我们处于动态设置，同一个球可能被多次插入、删除和重新插入。

首先注意，在仅插入的设置中，通过简单地使用**均匀探测**（uniform probing）——它将每个 $h_i(x)$ 设为随机的，并将每个球 x 放入序列 $h_1(x), h_2(x), \dots$ 中的第一个可用槽位——很容易实现平均探测复杂度 $O(\delta^{-1})$ 。然而，在动态设置中，据我们所知尚没有任何已知的桶分配方案能够实现 $\text{poly}(\delta^{-1})$ 的平均探测复杂度（例如，均匀探测仅在随机删除设置中被成功分析 [40]，而不移动物素的线性探测的分析仍然是一个开放问题 [52]）。

我们现在构造一个实现平均探测复杂度 $\text{poly}(\delta^{-1})$ 的桶分配方案。

定理 13. 假设 $\delta = 1/n^{o(1)}$ 。存在一种桶分配方案，支持任意球插入/删除/重新插入，并保证系统中每个球的期望探测复杂度为 $O(\text{poly}(\delta^{-1}))$ 。

证明. 考虑一个具有 n 个槽位和负载因子 $1-\delta$ 的可变大小微型指针解引用表。对每个球 x 和每个 $i \in \mathbb{N}$ ，定义 $h_i(x) = \text{Dereference}(x, i)$ 。为将球 x 分配到桶中，我们调用函数 $i = \text{Allocate}(x)$ ，并将 x 放入桶 $h_i(x) = \text{Dereference}(x, i)$ 。为删除球 x ，我们调用 $\text{Free}(x, i)$ 以释放解引用表中的相应槽位。

令 $c > 0$ 为某个足够小的正常数。由定理 2，每个球 x 被分配到桶 $h_i(x)$ ，其中 i （即由 $\text{Allocate}(x)$ 返回的微型指针）为

$$O(\log \delta^{-1} + P)$$

比特，其中 P 为满足 $\Pr[P \geq j] \leq O(2^{-2^{c j}})$ 的随机变量。由此得出 $\Pr[i \geq \text{poly}(\delta^{-1}) \cdot k] \leq O(2^{-2^{c \log k}}) = O(2^{-k^c})$ ，因此每个球 x 的期望探测复杂度为 $\text{poly}(\delta^{-1})$ 。□

8 致谢

本研究部分由 NSF 资助 CSR-1938180、CCF-2106999、CCF-2118620、CCF-2118832、CCF-2106827、CCF-1725543、CSR-1763680、CCF-1716252 和 CNS-1938709 以及 NSF GRFP 奖学金和 Fannie and John Hertz 奖学金支持。

本研究还部分由美国空军研究实验室和美国空军人工智能加速器赞助，并在合作协议编号 FA8750-19-2-1000 下完成。本文档中包含的观点和结论是作者的观点和结论，不应被解释为代表美国空军或美国政府的官方政策（无论是明示还是暗示）。美国政府被授权为政府目的复制和分发重印本，不论本文中的任何版权声明。

9 参考文献

- [1] Amd64 architecture programmer's manual volume 2: System programming. <https://www.amd.com/system/files/TechDocs/24593.pdf>. Accessed: 07/04/2021.
- [2] Intel 64 and ia-32 architectures software developer's manual combined volumes 3a, 3b, 3c, and 3d: System programming guide. <https://software.intel.com/content/www/us/en/develop/download/intel-64-and-ia-32-architectures-sdm-combined-volumes-3a-3d.html>. Accessed: 07/04/2021.
- [3] Google's Abseil C++ library. <https://abseil.io/>. Accessed: 2020-11-06.
- [4] George M Adel'son-Vel'skii and Evgenii Mikhailovich Landis. An algorithm for organization of information. In *Doklady Akademii Nauk*, volume 146, pages 263–266. Russian Academy of Sciences, 1962.
- [5] Stephen Alstrup, Gerth Brodal, and Theis Rauhe. Optimal static range reporting in one dimension. In *Proceedings of the Thirty-Third Annual ACM Symposium on Theory of Computing (STOC)*, pages 476–482, 2001.
- [6] Cecilia R. Aragon and Raimund Seidel. Randomized search trees. In *Proceedings of the 30th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 540–545, 1989.
- [7] Yuriy Arbitman, Moni Naor, and Gil Segev. De-amortized cuckoo hashing: Provable worst-case performance and experimental results. In *International Colloquium on Automata, Languages and Programming (ICALP)*, pages 107–118, Berlin, Heidelberg, 2009.
- [8] Yuriy Arbitman, Moni Naor, and Gil Segev. Backyard cuckoo hashing: Constant worst-case operations with a succinct representation. In *Proceedings of the 51st Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 787–796, 2010.
- [9] Vladimir L'vovich Arlazarov, Yefim A Dinitz, MA Kronrod, and Igor Aleksandrovich Faradzhev. On economical construction of the transitive closure of an oriented graph. In *Doklady Akademii Nauk*, volume 194, pages 487–488. Russian Academy of Sciences, 1970.
- [10] Michael A. Bender, Abhishek Bhattacharjee, Alex Conway, Martin Farach-Colton, Rob Johnson, William Kuszmaul, Don Porter, Guido Tagliavini, Janet Vorobyeva,

and Evan West. Paging and the address-translation problem. In *Proc. 32nd ACM Symposium on Parallelism in Algorithms and Architectures (SPAA)*, July 2021.

[11] Michael A Bender, Alex Conway, Martin Farach-Colton, William Kuszmaul, and Guido Tagliavini. All-purpose hashing. arXiv preprint arXiv:2109.04548, 2021.

[12] Michael A. Bender, Martin Farach-Colton, Mayank Goswami, Rob Johnson, Samuel McCauley, and Shikha Singh. Bloom filters, adaptivity, and the dictionary problem. In *Proceedings of the 59th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 182–193, Paris, France, October 2018.

[13] Michael A. Bender, Martin Farach-Colton, John Kuszmaul, William Kuszmaul, and Mingmou Liu. On the optimal time/space tradeoff for hash tables, 2021.

[14] Ioana Oriana Bercea and Guy Even. Fully-dynamic space-efficient dictionaries and filters with constant number of memory accesses. CoRR, abs/1911.05060, 2019.

[15] Ioana Oriana Bercea and Guy Even. A space-efficient dynamic dictionary for multisets with constant time operations. CoRR, abs/2005.02143, 2020.

[16] Joshimar Cordova and Gonzalo Navarro. Simple and efficient fully-functional succinct trees. *Theor. Comput. Sci.*, 656(PB):135–145, December 2016.

[17] cppreference std::unordered_map. https://en.cppreference.com/w/cpp/container/unordered_map. Accessed: 2020-11-06.

[18] gcc-mirror/gcc libstdc++-v3 unordered_map.h. https://github.com/gcc-mirror/gcc/blob/master/libstdc%2B%2B-v3/include/bits/unordered_map.h. Accessed: 2020-11-06.

[19] cppreference std::unordered_set. https://en.cppreference.com/w/cpp/container/unordered_set. Accessed: 2020-11-06.

[20] gcc-mirror/gcc libstdc++-v3 unordered_set.h. https://github.com/gcc-mirror/gcc/blob/master/libstdc%2B%2B-v3/include/bits/unordered_set.h. Accessed: 2020-11-06.

[21] cppreference std::map. <https://en.cppreference.com/w/cpp/container/map>. Accessed: 2020-11-06.

[22] gcc-mirror/gcc libstdc++-v3 stl_map.h. https://github.com/gcc-mirror/gcc/blob/master/libstdc%2B%2B-v3/include/bits/stl_map.h. Accessed: 2020-11-06.

[23] cppreference std::set. <https://en.cppreference.com/w/cpp/container/set>. Accessed: 2020-11-06.

[24] gcc-mirror/gcc libstdc++-v3 stl_set.h. https://github.com/gcc-mirror/gcc/blob/master/libstdc%2B%2B-v3/include/bits/stl_set.h. Accessed: 2020-11-06.

[25] Pooya Davoodi, Rajeev Raman, and Srinivasa Rao Satti. On succinct representations of binary trees. *Mathematics in Computer Science*, 11:177–189, 2017.

- [26] Erik D Demaine, Friedhelm Meyer auf der Heide, Rasmus Pagh, and Mihai Patrascu. De dictionariis dynamicis pauco spatio utentibus. In *Latin American Symposium on Theoretical Informatics (LATIN)*, pages 349–361. Springer, 2006.
- [27] Martin Dietzfelbinger and Rasmus Pagh. Succinct data structures for retrieval and approximate membership. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 385–396. Springer, 2008.
- [28] Martin Dietzfelbinger and Stefan Walzer. Constant-time retrieval with $o(\log m)$ extra bits. In *36th International Symposium on Theoretical Aspects of Computer Science (STACS 2019)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik, 2019.
- [29] Martin Dietzfelbinger and Christoph Weidling. Balanced allocation and dictionaries with tightly packed constant size bins. In *International Colloquium on Automata, Languages, and Programming (ICALP)*, pages 166–178. Springer, 2005.
- [30] Martin Dietzfelbinger and Philipp Woelfel. Almost random graphs with simple hash functions. In *Proceedings of the Thirty-Fifth Annual ACM Symposium on Theory of Computing (STOC)*, pages 629–638, 2003.
- [31] Facebook’s F14 hash table. <https://engineering.fb.com/2019/04/25/developer-tools/f14/>. Accessed: 2020-11-06.
- [32] Arash Farzan and J. Ian Munro. Succinct representation of dynamic trees. *Theoretical Computer Science*, 412(24):2668–2678, 2011.
- [33] Dimitris Fotakis, Rasmus Pagh, Peter Sanders, and Paul G. Spirakis. Space efficient hash tables with worst case constant access time. *Theory of Computing Systems*, 38(2):229–248, December 2005.
- [34] Gianni Franceschini and Roberto Grossi. Optimal worst-case operations for implicit cache-oblivious search trees. In *Workshop on Algorithms and Data Structures (WADS)*, pages 114–126. Springer, 2003.
- [35] Gaston H. Gonnet and Per-Ake Larson. External hashing with limited internal storage. *J. ACM*, 35(1):161–184, 1988.
- [36] Leonidas J. Guibas and Robert Sedgewick. A dichromatic framework for balanced trees. In *Proceedings of the 19th Annual Symposium on Foundations of Computer Science (FOCS)*, pages 8–21, 1978.
- [37] Takao Gunji and E Goto. Studies on hashing part-1: A comparison of hashing algorithms with key deletion. *J. Information Processing*, 3(1):1–12, 1980.
- [38] Don Knuth. Notes on "open" addressing, 1963.
- [39] Donald E. Knuth. *The Art of Computer Programming, Volume III: Sorting and Searching*. Addison-Wesley, 1973.
- [40] Per-Ake Larson. Analysis of uniform hashing. *J. ACM*, 30(4):805–819, 1983.

- [41] Per-Ake Larson. Linear hashing with separators – A dynamic hashing scheme achieving one-access retrieval. *ACM Trans. Database Syst.*, 13(3):366–388, 1988.
- [42] Per-Ake Larson and Ajay Kajla. File organization: Implementation of a method guaranteeing retrieval in one access. *Commun. ACM*, 27(7):670–677, 1984.
- [43] Mingmou Liu, Yitong Yin, and Huacheng Yu. Succinct filters for sets of unknown sizes. In *47th International Colloquium on Automata, Languages, and Programming (ICALP)*, volume 168 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 79:1–79:19. Schloss Dagstuhl–Leibniz-Zentrum fur Informatik, 2020.
- [44] Michael Molloy and Bruce Reed. *Graph coloring and the probabilistic method*. New York: Springer, 23:1329–356, 2002.
- [45] J. Ian Munro, Venkatesh Raman, and Adam J. Storm. Representing dynamic binary trees succinctly. In *Proceedings of the Twelfth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, pages 529–536, USA, 2001. Society for Industrial and Applied Mathematics.
- [46] Gonzalo Navarro and Kunihiro Sadakane. Fully functional static and dynamic succinct trees. *ACM Trans. Algorithms*, 10(3), May 2014.
- [47] Anna Pagh and Rasmus Pagh. Uniform hashing in constant time and optimal space. *SIAM Journal on Computing*, 38(1):85–96, 2008.
- [48] Rasmus Pagh and Flemming Friche Rodler. Cuckoo hashing. In *European Symposium on Algorithms (ESA)*, pages 121–133. Springer, 2001.
- [49] M. Patrascu. Succincter. In *Proceedings of the 49th Annual IEEE Symposium on Foundations of Computer Science (FOCS)*, pages 305–313, 2008.
- [50] W Wesley Peterson. Addressing for random-access storage. *IBM Journal of Research and Development*, 1(2):130–146, 1957.
- [51] Rajeev Raman and Satti Srinivasa Rao. Succinct dynamic dictionaries and trees. In *Proceedings of the 30th International Conference on Automata, Languages and Programming (ICALP)*, pages 357–368. Springer-Verlag, 2003.
- [52] Peter Sanders. Hashing with linear probing and referential integrity. arXiv preprint arXiv:1808.04602, 2018.
- [53] Raimund Seidel and Cecilia R. Aragon. Randomized search trees. *Algorithmica*, 16(4/5):464–497, 1996.
- [54] Daniel Dominic Sleator and Robert Endre Tarjan. Self-adjusting binary search trees. *J. ACM*, 32(3):652–686, 1985.
- [55] Berthold Vocking. How asymmetry helps load balancing. *J. ACM*, 50(4):568–589, July 2003.
- [56] Philipp Woelfel. Asymmetric balanced allocation with simple hash functions. In

Proceedings of the Seventeenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA), pages 424–433. ACM Press, 2006.

本文译自 *arXiv:2111.12800v1 [cs.DS]*